

Multi-Level Graceful Degradation for Mixed-Criticality Scheduling

Review copy, assembled from three previously separate documents (`task_model.tex`, `evaluation_methodology.tex`, `mode_optimization.tex` – each still compiles independently via the project Makefile; this file is the combined reading order). Highlighted boxes flag specific points for review (**REVIEW**) and open decisions for future work (**DECISION**). Section 23 near the end collects items that do not attach to one location. Remove every `\reviewnote/\decisionnote` call (and their definitions, near the top of the source) before any external submission.

Abstract

Mixed-criticality (MC) scheduling improves on worst-case pessimism by certifying high-criticality (HI) tasks under an optimistic assumption during normal operation and falling back to a pessimistic one if that assumption is violated. Across three decades of MC scheduling research, that fallback has almost always been a single, abrupt step: some or all low-criticality (LO) work is abandoned at once, and the literature’s refinements have targeted *when* that step fires, not what happens once it does. Graceful degradation – responding in proportion to severity rather than as one cliff-edge – is well established elsewhere in dependable systems design, but has not been brought into MC scheduling with the same analytical rigor the two-level case already has. This document presents a severity-indexed ladder of degradation levels whose safety is certified by the same response-time analysis used for classic two-level Adaptive MC (AMC) scheduling, generalised rather than replaced; the evaluation methodology and metrics used to assess it; the staged empirical protocol used to choose how the ladder should be configured, executed on real simulation data rather than merely proposed; and, following directly from what that protocol found, a further result on *exit* timing – when a degraded level should be left, not just entered – that turned out to be the largest single effect measured in this line of work. Results, methods, and a small number of open decisions are reported together, on the position that a reviewer should be able to see the whole argument in one document rather than reconstruct it from separate files.

1 Introduction

Safety-critical systems increasingly host tasks of different criticality on the same platform – functions whose failure could contribute to an accident alongside functions whose failure is merely inconvenient. Certifying every task to the same, conservative worst-case execution time (WCET) bound wastes most of the processor most of the time, since the bound a certification authority will accept for a high-criticality (HI) function is typically far more pessimistic than the execution time that function actually needs in normal operation. Vestal’s original formulation of mixed-criticality (MC) scheduling [20] exploits this directly: certify HI-criticality tasks against their pessimistic bound, but *schedule* the system, in normal operation, against a less pessimistic one – and provide a fallback guarantee for the case where a HI-criticality job actually needs more than that optimistic budget. The field this founded is now large enough to have its own survey [4] and its own recurring workshop, and the question of what the fallback should look like, and how to justify it to a certification authority, has been treated as a first-class problem in its own right rather than an afterthought [9].

The fallback has almost always been a single, abrupt step. The dominant formulation – Adaptive MC (AMC), and the runtime co-design refinements this project’s own prior work contributes [3] – has one normal mode and one degraded mode: a HI-criticality job reaching a response-time threshold trips a single transition, after which low-criticality (LO) work is abandoned, either entirely or via whatever conservative rule the exit protocol specifies. Bailout protocols make the same shape more literal still, dropping every LO-criticality task at the instant of detection rather than allowing any grading at all [1, 2]. Subsequent work in the field has continued to refine this picture – justifying more precisely how much LO-criticality service is actually lost [14], and asking how robust the two-level guarantee is under realistic assumptions [5] – but

in every case the refinement targets *when* the abrupt step fires or how quickly the system returns from it, not what the step itself looks like. Whether a HI-criticality overrun is barely over budget or catastrophically over budget, the LO-criticality response today is the same: gone.

Graceful degradation is not a new idea; it is simply new to this corner of scheduling theory. Responding to a fault or an overload in proportion to its severity, rather than as one discrete fallback, has been a recognised attribute of dependable systems since Laprie’s foundational terminology for the field [13], and a substantial, independent line of work in fault-tolerant and distributed embedded systems has built and evaluated staged degradation directly: fail-stop processors as a composable building block for a graded response [16], general architectural frameworks for graceful degradation in distributed embedded systems [18], product-family approaches to the same problem [15], and task-allocation algorithms explicitly optimised against a utility model of how the system degrades as components fail [6]. None of this is drawn from MC scheduling – it predates the field’s current form by decades in some cases – and none of it carries a real-time response-time guarantee of the kind MC scheduling depends on. It establishes something narrower but still useful here: that staged, proportionate degradation is a workable, well-studied design pattern in systems that must keep working under fault, not a speculative one.

Contribution. This document brings graceful degradation into MC scheduling with the same analytical grounding the two-level case already has, rather than as a heuristic utility function layered on top of it. The mechanism is a severity-indexed ladder of intermediate degradation levels, each protected by the *same* response-time analysis already used to certify the two-level guarantee – generalised across levels, not replaced by a new argument per level (Sections 2 and 7). Three things follow from taking that route seriously rather than assuming it works: a principled way to decide *how many* levels and where to place them, settled empirically rather than by convention (Section 15–Section 19); a full-scale empirical result comparing the scheme against two-level AMC across utilisation and deadline regimes (Section 21); and a second, unanticipated result that the same machinery makes answerable – not just when to enter a degraded level, but how gracefully to *leave* one, which turns out to be the single largest effect measured anywhere in this document (Section 21.5).

Structure of this document. Section 2–Section 10 present the task model, the safety argument for the graded ladder, and the two operating points the model admits. Section 11–Section 14 present the evaluation methodology: how task sets are generated, the metrics and objective function used to score a configuration, and the simulation parameters held fixed throughout. From Section 15 onward the document turns to the optimisation question this line of work exists to answer – how the ladder should be configured, and, following from what that investigation found, how it should be exited – reporting the staged protocol, its results, and what they do and do not establish. This is a review draft: specific points are flagged inline as they arise, and Section 23 near the end collects the small number of items that concern the document as a whole.

2 Design Rationale

This document presents the formal task model used in our research on multi-mode mixed-criticality (MC) scheduling. The model extends traditional two-mode Adaptive Mixed-Criticality (AMC) scheduling to support multiple criticality levels, enabling graded degradation rather than a single normal/degraded transition.

Key analysis principle. Every degradation level has its own execution budget, not just normal and fully-degraded. What makes the scheme analysable is not that only two budgets exist – they do not – but that each level’s worst-case execution time (WCET) budget is converted, via the *same* response-time analysis used for classic two-level AMC, into a worst-case response time (WCRT) that is monitored at run time. Reaching that WCRT is what triggers the transition into that level. Safety is preserved because each level’s operating point – its budget together with the set of LO-criticality tasks it keeps running – is checked at design time against exactly this analysis, generalised across levels rather than restricted to two (Section 7).

What this revision adds. Three changes, each motivated by a worked example in Section 3 and each with a consequence that is not cosmetic:

1. A *shed-aware* response time $R_i(\chi, S, s)$ (Section 5.4) that charges a shed task for the jobs it released before it stopped. It specialises to the normal-mode bound, to the severity trigger, and to AMC-rtb exactly, so it subsumes rather than replaces them – and it makes the instant of shedding an argument of the analysis, which is what lets the analysis prefer shedding early.

2. An *allocated criticality level* ℓ_i (Section 4.3) ordering tasks within the LO-criticality tier as well as across the HI/LO divide, because response-time analysis certifies which sets are safe to shed but cannot rank the safe ones.
3. A *generalised trigger* (Section 6.2) under which any task a rung would protect may demand that rung, with a parameter x_{LO} bounding how deep a LO-criticality task may reach. This detects overruns that the two-level trigger provably cannot see, at the cost of a containment guarantee that must be restated rather than assumed.
4. *Deadline termination* and the JNC objective (Sections 8.3 and 8.4). A LO-criticality job that cannot finish in time is discarded at its deadline rather than allowed to complete late, since a stale result may be worse than none. This removes the LO-criticality deadline obligation entirely – there is nothing left to miss – which turns clause 2 of the admissibility criterion from a constraint into an objective.

That last change yields **two distinct operating points**, both reported in Section 9 because they suit different certification appetites. The *conservative* point keeps clause 2, so every retained LO-criticality task is certified to complete and LDM is zero by analysis; it needs no new execution model, being sound under the original two-level semantics. The *termination* point relaxes clause 2 for more retained work, at the cost of accepting that a job may be discarded having run. Across the properly-powered regime map of 32 configurations (mode_optimization.tex, “Stage Results”; $U \in \{0.6, \dots, 0.9\}$, both deadline regimes), both points beat two-level AMC-RA in *every* cell tested – +2.4% to +12.4% for the conservative point, +3.5% to +27.3% for termination – and the gain widens as the regime hardens rather than collapsing. An earlier draft of this document reported a loss in the hardest regime tested; that claim came from an underpowered pilot and did not reproduce (Section 9).

Two findings are consequential and negative, and are reported as such. *Progressive* shedding cannot be certified on 47% of an otherwise schedulable population once shedding is charged honestly, while shedding the same tasks at the shallowest rung is always certifiable and never sheds more (Section 7); grading therefore belongs in the budgets, not the drop sets. And the generalised trigger, though it targets a blind spot that swallows 87–89% of all overruns, changes the service ratio by under 0.05 percentage points, because a single drop set leaves escalation with nothing further to shed (Section 6.2).

The model builds upon the foundational work presented in [3].

3 Motivating Examples

Three situations that classic two-level AMC handles badly, each motivating one feature of the model that follows. They are worked out concretely rather than described, because in every case the argument turns on the gap between a worst-case response-time *bound* and the response time actually observed. Every schedule and every number below is reproduced by the accompanying implementation.

3.1 Example 1: deferring the shed starves the task it protects

Task	κ_i	Priority	$T_i = D_i$	C_i^{lo}	C_i^{hi}
τ_1	LO	0 (high)	10	4	–
τ_2	HI	1	50	8	20
τ_3	LO	2 (low)	100	5	–

Here $R_2(0) = R_2^{lo} = 16$ and $R_2(1) = 36$, so the shallowest rung fires at $t = 16$ and the deepest at $t = 36$. Suppose τ_2 exhibits HI-criticality behaviour and executes its full $C_2^{hi} = 20$, with every task released at $t = 0$:

(a) nothing shed	1111222222111122222211112222221111223333				
(b) τ_1 shed at $t = 16$	111122222211112222222222223333.....				
	0	10	20	30	40

In (a) τ_2 completes at $t = 36$: it is preempted for four ticks in every ten by τ_1 , so it accrues execution at 6/10 of real time and takes 36 ticks of wall-clock to consume 20 ticks of budget. It therefore *reaches the deepest rung at the very instant it finishes*, and the deepest rung abandons every LO-criticality task – τ_3 included, which was never the problem.

In (b) the single higher-priority LO-criticality task is shed at the *shallowest* rung, at $t = 16$. τ_2 then runs uninterrupted and completes at $t = 28$, so $R_2(1) = 36$ is never reached and the deepest rung never fires. The cost is four abandoned releases of τ_1 ; the saving is that τ_3 is not abandoned at all and the system never fully degrades.

The point is not that τ_2 needed a larger budget – it was entitled to C_2^{hi} throughout, and (b) does not give it one tick more. It is that a run-time trigger measures *elapsed time* while a job’s progress is *execution time*, and higher-priority LO-criticality work drives a wedge between the two. Shedding that work early converts elapsed time into progress. This is why Section 5.4 makes the shed instant an argument of the response-time bound, and why Section 7 concludes that shedding belongs at the shallowest rung.

3.2 Example 2: safety does not determine which tasks to keep

Task set seed 7 of the generator ($n = 6$, $U = 0.7$, $CF = 2$), priorities by Audsley’s algorithm:

Task	κ_i	Priority	$T_i = D_i$	C_i^{lo}	C_i^{hi}
τ_5	LO	0	398	64	–
τ_4	LO	1	282	38	–
τ_3	HI	2	3559	164	423
τ_1	HI	3	1779	2	5
τ_2	HI	4	6229	496	3605
τ_6	LO	5	5586	1547	–

Enumerating every subset of $\{\tau_4, \tau_5, \tau_6\}$ against the admissibility criterion of Section 7 yields exactly two *minimal* admissible drop sets:

$$\{\tau_4\} \text{ (keeping } \tau_5, \tau_6) \quad \text{and} \quad \{\tau_5\} \text{ (keeping } \tau_4, \tau_6).$$

Two-level AMC sheds all three. The graded scheme can keep two – but *which* two is not a question the analysis answers: both choices are equally safe, and both retain the same number of tasks. τ_4 and τ_5 are interchangeable to response-time analysis and are not interchangeable to the application. The shedding heuristics of Section 7 disagree accordingly.

This is the gap an *allocated criticality level* fills. Response-time analysis certifies that a set is safe to shed; it cannot rank the safe sets. Section 4.3 therefore replaces the binary $\kappa_i \in \{\text{HI}, \text{LO}\}$ with a level ℓ_i that orders tasks within the LO-criticality tier as well as across the divide.

3.3 Example 3: an overrun the two-level trigger cannot see

Task	κ_i	Priority	$T_i = D_i$	C_i^{lo}	C_i^{hi}
τ_1	LO	0 (high)	10	3	–
τ_2	HI	1	20	2	4
τ_3	LO	2 (low)	40	5	–

$R_2^{\text{lo}} = 5$ and $R_3^{\text{lo}} = 10$. Let τ_2 and τ_3 be released at $t = 0$, let τ_1 ’s next release fall at $t = 5$ (its previous job ran in $[-5, -2]$), so the processor is idle at $t = -1$ and τ_3 ’s priority level-3 busy period genuinely begins at $t = 0$), and let τ_2 execute its full $C_2^{\text{hi}} = 4$:

2222	3111	3333	...	111..
0123	4567	8901	234	56789

τ_2 *overruns* – it executes C_2^{hi} , twice its normal budget – yet completes at $t = 4$, before its own trigger $R_2^{\text{lo}} = 5$. Under two-level AMC, and under every rung of the ladder as stated in Section 6.2, **the system never degrades at all**, because only a HI-criticality task may pull the trigger and this one beat its own bound. It beat it because R_2^{lo} budgets for a full interference term from τ_1 that did not materialise in this release.

The overrun did not vanish; it landed on τ_3 , which at $t = 10 = R_3^{\text{lo}}$ has executed 3 of its 5 ticks and completes at $t = 12$. A LO-criticality task has slack of its own, but it is the last to absorb the error and the first to run out.

So τ_3 can detect an overrun that τ_2 concealed. Letting it act on that detection – shedding a lower-ranked LO-criticality task to protect itself – is the second mechanism this model adds (Section 6.2). It comes at a price that must be stated plainly: the scheme now degrades at instants when two-level AMC does not, so the guarantee that it never abandons more LO-criticality work than AMC-RH is *false* in general. This example is the counterexample, and Section 6.2’s parameter x_{LO} is the dial that trades one against the other.

4 Task Model

4.1 Basic Task Properties

A task set $\tau = \{\tau_1, \tau_2, \dots, \tau_n\}$ consists of n independent, sporadically arriving tasks. Each task τ_i is characterized by:

Table 1: Task Properties

Symbol	Name	Description
$T_i \in \mathbb{N}^+$	Period	Minimum inter-arrival time (ticks)
$D_i \in \mathbb{N}^+$	Deadline	Relative deadline ($D_i = T_i$ for implicit deadlines)
$C_i^{\text{lo}} \in \mathbb{N}^+$	Normal budget	Execution budget in normal mode
$C_i^{\text{hi}} \in \mathbb{N}^+$	HI execution budget	Execution budget once fully degraded ($C_i^{\text{hi}} \geq C_i^{\text{lo}}$)
$\text{BCET}_i \in \mathbb{N}^+$	Best-case execution time	$\text{BCET}_i \leq C_i^{\text{lo}}$

4.2 Severity and Per-Level Budgets

Every degradation level has its own execution budget. Rather than name each level’s budget independently, we index it by a *severity* $\chi \in [0, 1]$ and interpolate:

$$C_i(\chi) = \max(C_i^{\text{lo}}, C_i^{\text{lo}} + \chi \cdot (C_i^{\text{hi}} - C_i^{\text{lo}}))$$

so that $C_i(0) = C_i^{\text{lo}}$, $C_i(1) = C_i^{\text{hi}}$, and χ between the two endpoints gives an intermediate budget. The outer max is not decorative: it is what guarantees $C_i(\chi) \geq C_i^{\text{lo}}$ for every χ , which in turn is what makes the response times derived from it never earlier than the normal-mode response time (Section 5.3 below) – rounding must never be allowed to violate it. Linear interpolation is a design choice, not a derivation: every property in this document holds for any interpolation that is monotone in χ and never below C_i^{lo} , so a better-motivated member of that family is an open question, not a constraint this analysis depends on.

4.3 Criticality Assignment

Each task carries the conventional binary criticality

$$\kappa_i \in \{\text{HI}, \text{LO}\}$$

with $\tau^{\text{HI}} = \{\tau_i \in \tau \mid \kappa_i = \text{HI}\}$ and $\tau^{\text{LO}} = \{\tau_i \in \tau \mid \kappa_i = \text{LO}\}$. This governs the *budget*: only a HI-criticality task has $C_i^{\text{hi}} > C_i^{\text{lo}}$, and only a HI-criticality task has a deadline obligation once fully degraded.

It does not, however, say anything about *order of sacrifice* within the LO-criticality tier, and Section 3.2 shows that response-time analysis alone cannot supply one: a task set there admits two minimal drop sets of

equal size, both provably safe, differing only in which LO-criticality task survives. So each task additionally carries an *allocated criticality level*

$$\ell_i \in \{1, \dots, k-1\},$$

read as: τ_i runs at levels $L_1, \dots, L_{\ell_i}$ and is abandoned at L_{ℓ_i+1} . Every HI-criticality task has $\ell_i = k-1$, since it is never abandoned. The allocation induces the drop sets directly,

$$S_x = \{i \mid \ell_i < x\},$$

which are nested by construction, so choosing the allocation and choosing the ladder of drop sets are the same act. Allocating those levels well is the optimisation this work exists to study; the present document fixes only what makes an allocation *admissible* (Section 7).

4.4 Utilisation

The utilisation of task τ_i under different criticality modes is:

$$U_i^{\text{lo}} = \frac{C_i^{\text{lo}}}{T_i}, \quad U_i^{\text{hi}} = \begin{cases} \frac{C_i^{\text{hi}}}{T_i} & \text{if } \kappa_i = \text{HI} \\ 0 & \text{if } \kappa_i = \text{LO} \end{cases}$$

The total LO-criticality utilisation is $U = \sum_{i=1}^n U_i^{\text{lo}}$. The HI-criticality utilisation factor (CF) relates aggregate utilisations:

$$U^{\text{hi}} = \sum_{\tau_i \in \tau^{\text{HI}}} U_i^{\text{hi}}, \quad \text{where } U^{\text{hi}} = CP \cdot CF \cdot U$$

Here $CP = |\tau^{\text{HI}}|/n$ is the criticality proportion.

5 Response Time Analysis

This section presents the standard fixed-priority response-time analysis, then generalises it to a severity χ . It is placed before the degradation model (Section 6) because that model's trigger points and safety argument are both stated in terms of the response times defined here.

5.1 Normal-Mode Response Time

For fixed-priority preemptive scheduling, the response time of task τ_i under normal-mode budgets is the least fixed point of

$$R_i^{\text{lo}} = C_i^{\text{lo}} + \sum_{\tau_j \in \text{hp}(i)} \left\lceil \frac{R_i^{\text{lo}}}{T_j} \right\rceil C_j^{\text{lo}}$$

where $\text{hp}(i)$ is the set of tasks with higher priority than τ_i . **Requirement R1**, that a task set meets every deadline when every job complies with its LO budget, holds exactly when $R_i^{\text{lo}} \leq D_i$ for every $\tau_i \in \tau$.

5.2 Classic Two-Level Schedulability (AMC-rtb)

The design-time question "can this task set tolerate full degradation at all?" is answered independently of any run-time trigger, by the AMC-rtb test. For a HI-criticality task τ_i , the response time once every LO-criticality task has been abandoned is the least fixed point of

$$R_i^{\text{hi}} = C_i^{\text{hi}} + \sum_{\tau_j \in \text{hp}_{\text{HI}}(i)} \left\lceil \frac{R_i^{\text{hi}}}{T_j} \right\rceil C_j^{\text{hi}} + \sum_{\tau_k \in \text{hp}_{\text{LO}}(i)} \min\left(\left\lceil \frac{R_i^{\text{hi}}}{T_k} \right\rceil, \left\lceil \frac{R_i^{\text{lo}}}{T_k} \right\rceil\right) C_k^{\text{lo}}$$

where $\text{hp}_{\text{HI}}(i)$ and $\text{hp}_{\text{LO}}(i)$ are the higher-priority HI-criticality and LO-criticality tasks, respectively. The $\min(\cdot, \cdot)$ term bounds the interference from a higher-priority LO-criticality task by the number of its jobs released before degradation begins, since every LO-criticality task is abandoned on release once fully degraded.

The switch occurs at or before R_i^{lo} within τ_i 's busy period, so τ_k releases at most $\lceil R_i^{\text{lo}}/T_k \rceil$ jobs; the first argument of the min is the sound tightening that interference over a window of length R_i^{hi} cannot exceed $\lceil R_i^{\text{hi}}/T_k \rceil$ jobs however early the switch occurs. A previous statement of this equation carried a spurious +1 on the second argument, which the implementation never had. A task set is AMC-rtb schedulable if $R_i^{\text{hi}} \leq D_i$ for every $\tau_i \in \tau^{\text{HI}}$ and $R_i^{\text{lo}} \leq D_i$ for every $\tau_i \in \tau$.

This is the two-level baseline this work compares against, and the degenerate case ($k = 2$, every LO-criticality task abandoned at the single trigger) that the general model in the next section must reduce to exactly.

5.3 Severity-Indexed Response Time

Generalise Section 5.1 by charging *every* task at severity χ rather than only at C_i^{lo} :

$$R_i(\chi) = C_i(\chi) + \sum_{\tau_j \in \text{hp}(i)} \left\lceil \frac{R_i(\chi)}{T_j} \right\rceil C_j(\chi)$$

using the per-task budgets of Section 4.2. If this recurrence exceeds D_i before converging, level χ is unreachable for task i and $R_i(\chi) = \infty$ – a threshold that never fires, rather than the partially-iterated value the recurrence had reached, which would depend on where iteration happened to stop and break the monotonicity property below.

$R_i(\chi)$ answers a different question from R_i^{hi} in Section 5.2: it is the response time *if every task, including every LO-criticality one, were uniformly behaving at severity χ with none yet abandoned*. That is deliberately more pessimistic than the true response time once a drop policy is in effect, and pessimism in this direction is what makes $R_i(\chi)$ safe to use as a run-time trigger: reaching it is evidence that observed behaviour is *at least* that severe, never a false alarm. R_i^{hi} , by contrast, models the true operating response time under the specific policy "abandon every LO-criticality task", and is a design-time schedulability test, not something monitored at run time.

Three properties hold for every task set, and are what the multi-level scheme's safety argument (Section 7) rests on. They are checked directly, not merely argued, in `tests/scheduling/test_severity.py`.

- (A) $R_i(0) = R_i^{\text{lo}}$ exactly – a level at severity 0 triggers at exactly the classic single AMC trigger point.
- (B) $\chi_a \leq \chi_b \implies R_i(\chi_a) \leq R_i(\chi_b)$ – the severity-to-response-time map is monotone, so a ladder of increasing severities produces a ladder of non-decreasing triggers, given the ∞ -saturation above.
- (C) $R_i(\chi) \geq R_i^{\text{lo}}$ for every χ – no severity level ever triggers earlier than the classic AMC trigger.

Property (C) is what an earlier fraction-based design ($\chi \cdot R_i^{\text{lo}}$ for $\chi < 1$, triggering *below* the classic threshold rather than at or above it) violated by construction: measured with no HI-criticality behaviour present anywhere, such a threshold at $0.9 \cdot R_i^{\text{lo}}$ fired 14,118 times across twelve task sets, abandoning 356 jobs with no fault present. Charging every task's own budget, rather than scaling the deadline, is what avoids this.

5.4 Shed-Aware Response Time

$R_i(\chi)$ above charges every LO-criticality task in full, as though none were ever shed. That is deliberately safe for a *trigger*, but it is far too pessimistic for an *obligation*: the whole point of a drop set is that the tasks in it stop releasing. Conversely, treating a shed task as vanishing the instant its level is entered – which is what the analysis in this document originally did – is optimistic, and unsound: abandonment is on release, so a job already in flight runs to completion and still interferes.

Both are corners of one recurrence. Let S be the set shed, and let s_k bound the instant, measured from the start of τ_i 's busy period, at which $\tau_k \in S$ stopped releasing. Then $R_i(\chi, S, s)$ is the least fixed point of

$$w = C_i(\chi) + \sum_{\tau_j \in \text{hp}(i) \setminus S} \left\lceil \frac{w}{T_j} \right\rceil C_j(\chi) + \sum_{\tau_k \in \text{hp}(i) \cap S} \min\left(\left\lceil \frac{w}{T_k} \right\rceil, \left\lceil \frac{s_k}{T_k} \right\rceil\right) C_k^{\text{lo}}.$$

The $\min(\cdot, \cdot)$ term is exactly the one AMC-rtb applies to the two-level case, generalised to a per-task shed instant. Three specialisations recover everything already defined, and are checked as an equality (not an inequality) over 240 HI-criticality tasks across 40 task sets:

Specialisation	Yields	Meaning
$R_i(0, \emptyset, \cdot)$	R_i^{lo}	Section 5.1
$R_i(\chi, \emptyset, \cdot)$	$R_i(\chi)$	Section 5.3, the trigger
$R_i(1, \tau^{\text{LO}}, R_i^{\text{lo}})$	R_i^{hi}	Section 5.2, AMC-rtb

Three monotonicity properties follow directly, and the third is what Section 3.1 illustrates:

(M1) $R_i(\chi, S, s)$ is non-decreasing in χ .

(M2) $R_i(\chi, S, s)$ is non-increasing in S under inclusion, because $C_k^{\text{lo}} = C_k(\chi)$ for every LO-criticality task.

(M3) $R_i(\chi, S, s)$ is non-decreasing in each s_k : **shedding earlier never hurts, and generally helps.**

Where the shed instant comes from. s_k is not free. The rung that sheds τ_k is entered no later than $R_i(\chi_{\ell_k+1})$ within τ_i 's own busy period, because if τ_i is still incomplete at that instant then τ_i fires the rung itself. The argument is available only to a task that is permitted to fire that rung – always true for a HI-criticality task, and true for a retained LO-criticality task exactly when $\ell_k + 1 \leq x_{\text{LO}}$ (Section 6.2). Where it is not available, $s_k = \infty$ and τ_k is charged in full, as though never shed.

That caveat is not a technicality: $R_i(\chi) = \infty$ for 33% of HI-criticality tasks at $\chi = 1$, so a task shed at the *deepest* rung frequently has no finite bound at all. This is what drives the conclusion of Section 7.

6 Multi-Level Degradation Model

6.1 Degradation Levels

The system has $k \geq 2$ levels $L_0, L_1, \dots, L_{k-1}$, each assigned a severity $0 = \chi_1 \leq \chi_2 \leq \dots \leq \chi_{k-1} \leq 1$ (level L_0 has no severity; it is the absence of any triggered level). Pinning $\chi_1 = 0$ is what makes L_1 trigger at exactly the classic AMC point (property A) and is required by the safety argument in Section 7; χ_{k-1} need not be 1, though setting it so recovers the classic two-level scheme's full protection at the deepest level.

Table 2: Degradation Level Structure

Level	Severity	Trigger Condition	LO Task Handling
L_0	–	None	All tasks run at C_i^{lo}
L_1	$\chi_1 = 0$	HI task reaches $R_i(0) = R_i^{\text{lo}}$	Drop set S_1 (Section 7)
L_2	χ_2	HI task reaches $R_i(\chi_2)$	Drop set $S_2 \supseteq S_1$
$\vdots$	$\vdots$	$\vdots$	$\vdots$
L_{k-1}	χ_{k-1}	HI task reaches $R_i(\chi_{k-1})$	Drop set $S_{k-1} \supseteq \dots \supseteq S_1$

Each level's operating budget for every task is $C_i(\chi_x)$ (Section 4.2): a task entering L_x because severity χ_x was reached is granted exactly the execution allowance that severity justifies, not immediately the full C_i^{hi} – that is what makes the degradation graded rather than binary. One consequence is worth stating plainly: with $\chi_1 = 0$, L_1 's operating budget equals L_0 's. Entering L_1 changes nothing about what any task is allowed to execute; it only marks that an overrun has been confirmed somewhere; and it must still ask whether any LO-criticality task needs to be shed to keep HI tasks safe at that (unchanged) budget, which Section 7 answers per level. This is the same principle Theorem 1 of the safety proof states for L_0 itself: protection begins no later than the classic trigger, never earlier.

6.2 Mode Transition Protocol

Classically, only a HI-criticality task may pull the trigger:

$$L_x \rightarrow L_{x+1} \quad \text{when } \exists \tau_i \in \tau^{\text{HI}} : \text{active job reaches } R_i(\chi_{x+1}).$$

Section 3.3 shows what that misses. A HI-criticality task can execute its full C_i^{hi} – a genuine overrun – and still complete before R_i^{lo} , because R_i^{lo} budgets for interference that a particular release need not receive. The overrun is then invisible to every rung, while the LO-criticality task that absorbed it runs out of slack.

The generalisation is to let *any* task that a rung would protect demand that rung. Using the allocated levels of Section 4.3:

$$\begin{aligned} L_x \rightarrow L_{x+1} & \quad \text{when } \exists \tau_i \text{ with } \ell_i \geq x+1 \text{ and } (\kappa_i = \text{HI or } x+1 \leq x_{\text{LO}}) : \text{active job reaches } R_i(\chi_{x+1}) \\ L_x \rightarrow L_y & \quad y < x \text{ on exit (direct or cascade, per protocol design)} \end{aligned}$$

A task never fires a rung that would abandon it ($\ell_i \geq x+1$), so the abandoned set is always a strict prefix of the criticality order.

The threshold is calculated, not searched. A LO-criticality task’s threshold for demanding rung x is its own $R_i(\chi_x)$ – the same quantity, computed the same way, as for a HI-criticality task. Soundness is the same one line: if every task had complied with $C(\chi_x)$, τ_i would have completed by $R_i(\chi_x)$, so reaching it is evidence that behaviour is at least that severe. Properties (A), (B) and (C) of Section 5.3 hold verbatim for LO-criticality tasks, so no rung fires earlier than the classic trigger and none fires with no fault present.

The mechanism is not vacuous even though $C_i(\chi) = C_i^{\text{lo}}$ for every LO-criticality task and every χ : $R_i(\chi)$ still grows with χ through the HI-criticality tasks in $\text{hp}(i)$. Measured over 199 task sets, 48% of LO-criticality tasks have at least two distinct finite thresholds (median spread 56% of the smallest), and 83% have a higher-priority LO-criticality task available to shed.

The parameter x_{LO} . It is the deepest rung a LO-criticality task may fire, and it is a genuine dial rather than a convenience. At $x_{\text{LO}} = 0$ only HI-criticality tasks trigger and the scheme provably abandons no more LO-criticality work than AMC-RH. Above 0 that containment is lost – Section 3.3 is the counterexample – and what is bought is protection for the higher-ranked LO-criticality tiers against overruns two-level AMC cannot detect. The safety consequences are set out in the accompanying safety proof; the trade is measured empirically rather than settled by argument.

Measured status of the mechanism. The blind spot Section 3.3 describes is not an edge case. Across 60 task sets and three seeds, at every overrun rate from $f_p = 10^{-3}$ to $f_p = 0.2$, 87–89% of **HI-criticality overruns fire no rung at all**: the job executes beyond C_i^{lo} and still completes before R_i^{lo} . At $f_p = 0.2$ that is 129,874 undetected overruns out of 145,769.

It does not follow that the mechanism pays, and measured over the same population it does not: enabling $x_{\text{LO}} = 1$ changes the service ratio by less than 0.05 percentage points and adds 20 level entries in 12,714. The reason is specific and worth stating, because it is not the reason one would guess.

Why it does not engage. The mechanism has been tested against three successive designs, each of which removed the previous explanation for its failure:

1. *Clause 2 enforced, one drop set.* A retained LO-criticality task is certified to complete, so none is ever at risk; and a shed task has $\ell_i = 0$ and may fire no rung. No task is both permitted to use the mechanism and in need of it.
2. *Clause 2 relaxed, one drop set.* Retained tasks are now best-effort, so they *can* be at risk – but with a single drop set, entering a deeper rung sheds nothing further, so escalating has no content. Of 386 terminations measured at $U = 0.7$ with tightened deadlines, 376 occurred while the system was *already degraded*.
3. *Intermediate ladder, genuine middle tiers.* S_1 at rung 1 and S_2 at rung 2, so escalation does shed something new and a guaranteed-tier task can act. LO-criticality tasks now demonstrably do fire rungs – level entries rise from 3,900 to 3,942 at $U = 0.70$. The service ratio falls from 0.9642 to 0.9640.

In the third design the mechanism is fully enabled and simply loses the exchange. Escalating early sheds the middle tier *immediately and with certainty*; the benefit – one struggling task completing that otherwise

would not – is rare and worth at most one job. Raising x_{LO} from 1 to 2 changes nothing further, so the result is not an artefact of how deep a LO-criticality task is permitted to reach.

This is a negative result about the mechanism, not about the trigger rule that supports it. The rule is retained in the model because it is what a different cost structure would activate: it costs nothing when $x_{\text{LO}} = 0$, and the blind spot it addresses is real and large even though closing it does not currently pay.

The trigger point for the deepest level, $R_i(\chi_{k-1})$, is written R_{trigger} where the level count is not otherwise relevant; with $\chi_{k-1} = 1$ it recovers the classic AMC trigger for full degradation.

7 Admissible Drop Sets and Safety

Retaining LO-criticality work at an intermediate level necessarily means absorbing interference that abandoning it would have avoided – so the drop set S_x at each level is not a free design choice, it is an obligation discharged by response-time analysis at design time.

Definition (admissible drop set). $S_x \subseteq \tau^{\text{LO}}$ is *admissible* at severity χ_x if, charging every task $C_i(\chi_x)$, charging each shed task its carry-in per Section 5.4 rather than treating it as absent, and every retained LO-criticality task at C_i^{lo} :

1. every HI-criticality task τ_i has response time $R_i \leq D_i$, and
2. every retained LO-criticality task $\tau_j \notin S_x$ has response time $R_j \leq D_j$.

Clause 1 is a constraint; clause 2 is an objective. Clause 1 is what certification requires and is always enforced. Clause 2 is not a safety obligation at all under Section 8.3: a retained LO-criticality job that cannot finish is terminated at its deadline, so it never delivers a late result – it delivers none, which is the same outcome as being abandoned on release and is counted identically by JNC. Requiring clause 2 therefore buys *certainty of service*, not safety.

It charges heavily for that certainty. A task the analysis cannot certify must be shed outright, so every LO-criticality task is either guaranteed to complete or never runs, with nothing in between – and it is the state in between where graded criticality actually lives. Over 199 non-trivial AMC-rtb-schedulable task sets ($n = 12$, $U = 0.7$, $CF = 2$), shedding at the shallowest rung with carry-in charged:

Table 3: Cost of enforcing clause 2

Clause 2	LO tasks shed	LO tasks retained
enforced	68.3%	31.7%
relaxed (default)	47.6%	52.4%
classic two-level	100.0%	0.0%

Relaxing it raises retention from 31.7% to 52.4% at no cost in HI-criticality safety, because clause 1 is untouched and a terminated job executes no more than it otherwise would.

An admissible set always exists whenever the task set passes AMC-rtb (Section 5.2): $S_x = \tau^{\text{LO}}$ reduces clause 1 to the AMC-rtb HI-mode test and vacates clause 2, recovering the classic two-level scheme exactly. So the question a drop policy answers is never *whether* a level is feasible, only how little must be shed – which is what makes the drop set an optimisation rather than a guess, and is addressed separately (drop-strategy research, out of scope for this document).

Where the shed must happen. Carry-in changes the design conclusion, not merely the numbers. A task shed at rung r stops releasing at $R_i(\chi_r)$ for that rung’s *trigger* severity, and by Section 5.4 that bound is infinite for 33% of HI-criticality tasks at $\chi = 1$. Where there is no bound the shed task is charged in full, which no amount of further shedding can repair – so a rung that first sheds a task deep in the ladder may be uncertifiable outright. Measured over 199 non-trivial AMC-rtb-schedulable task sets ($n = 12$, $U = 0.7$, $CF = 2$) with trigger severities $(0, 0.5, 1)$:

[†] over the certifiable subsample only, and therefore *not* comparable to the row below it: progressive shedding succeeds only on the easiest third of the population, which flatters its average. The like-for-like comparison

Table 4: Shedding progressively against shedding at the shallowest rung, under the default (relaxed clause 2)

Policy	Task sets uncertifiable	LO tasks shed
Progressive (each task at its own rung)	130 (65%)	36.0% [†]
Shed-early (one set, all at rung 1)	0 (0%)	47.6%
Classic two-level (single trigger)	0 (0%)	100%

is head to head on the 69 task sets where both policies are certifiable, and there shedding early sheds strictly less on 21, the same on 48, and more on **none**.

Shedding early is also feasible exactly when the task set passes AMC-rtb – shedding everything at R_i^{lo} is Section 5.2’s equation – so it never demands more than the classic test and never rescues a set the classic test rejects.

The conclusion is that **grading belongs in the budgets, not in the drop sets**. Progressive shedding buys nothing the analysis can certify and costs certifiability on two thirds of the population, which is the formal counterpart of Section 3.1: a shed deferred to a deep rung arrives too late to help the task it was meant to protect.

There is one exception, and it matters for Section 9. Once S_1 discharges clause 1 at *every* operating severity, a deeper rung may shed *more* without any risk to safety: a shed task’s charge, $\min(\lceil w/T_k \rceil, \lceil s_k/T_k \rceil)$, is at worst $\lceil w/T_k \rceil$, which is exactly what it would contribute unshed. So even an infinite shed instant cannot raise the interference above the already-certified level. Verified on all 199 task sets: extending the drop set at rung 2 never broke clause 1. Progressive shedding is thus safe whenever the *safety* obligation is discharged in full at the shallowest rung and the deeper rungs add only service – which is exactly how the intermediate policy of Section 9 is built.

That conclusion has a cost recorded in Section 6.2: a single drop set leaves no intermediate tiers, and without intermediate tiers escalation has nothing further to shed. Certifying progressive shedding – which needs the windowed two-phase analysis described below – is therefore the prerequisite for graded criticality levels being more than a two-way split.

Under the stricter criterion (clause 2 enforced) the same comparison gives 93 of 199 uncertifiable for progressive against 0 for shed-early, at 71.5% and 68.3% shed respectively; and under the original assumption that shedding takes effect instantaneously, 0.0% shed at $\chi \leq 0.25$, 10.0% at 0.50, 33.7% at 0.75 and 56.3% at 1.00. Both are retained only for comparison – the first buys service certainty rather than safety, the second is optimistic by the whole carry-in term.

Nesting. $S_1 \subseteq S_2 \subseteq \dots \subseteq S_{k-1}$ is required so that a level transition only ever adds to the drop set, keeping the run-time decision incremental. This holds automatically for any drop strategy that is a pure function of the task set and the remaining candidates (such greedy strategies shed the same sequence of tasks regardless of severity, so each level’s set is a prefix of one sequence, and prefixes nest) – it is not an extra assumption on top of the definition above.

Scope of this analysis. The carry-in term of Section 5.4 discharges what was previously the largest gap here: the bound now matches the simulator’s abandonment-on-release policy, under which a job already released when a level is entered runs to completion and still interferes. What remains unmodelled is the interference profile *before* a rung fires. The bound charges every task at the rung’s operating severity for the whole busy period, including the interval in which the system was still at a shallower rung and a lower severity would have sufficed. A windowed two-phase form – charging $C_j(\chi_{x-1})$ before the transition instant and $C_j(\chi_x)$ after, as AMC-rtb does for the LO-criticality term – would tighten this, and is the natural next step. It is a source of pessimism, not of unsoundness: every figure in Table 4 is an upper bound on what must be shed.

8 Job Execution Model

8.1 Execution Time Distribution

Each job’s execution time is drawn at release:

$$E_i \sim \begin{cases} \text{Uniform}(\text{BCET}_i, C_i^{\text{lo}}) & \text{if } \kappa_i = \text{LO} \\ \text{Uniform}(\text{BCET}_i, C_i^{\text{lo}}) & \text{if } \kappa_i = \text{HI, normal behaviour} \\ \text{Uniform}(C_i^{\text{lo}}, C_i^{\text{hi}}) & \text{if } \kappa_i = \text{HI, HI behaviour} \end{cases}$$

HI-criticality behaviour occurs with probability $FP = 1/N$ per job.

8.2 Execution Budget Enforcement

A job is constrained by its own worst-case budget, not by the level the system happens to be in:

$$\text{Actual execution} = \min(E_i, C_i^{\text{hi}}).$$

$C_i(\chi_x)$ is an *analysis* quantity – what the response-time bound charges at severity χ_x – and never a run-time throttle. The distinction is not cosmetic. Enforcing $C_i(\chi_x)$ at run time would cap a HI-criticality job at $C_i(\chi_1) = C_i^{\text{lo}}$ in the shallowest level, making an overrun physically impossible; no threshold could then ever be reached and the ladder would never leave L_0 . Implementing that reading confirmed it exactly: over a two-million-tick run with $f_p = 0.05$, 1,469 jobs were truncated and the system made zero level transitions.

For LO-criticality tasks $C_i^{\text{hi}} = C_i^{\text{lo}}$, so this is a real cap on them and a defensive one on HI-criticality tasks, matching the two-level model exactly.

8.3 Deadline Termination

A LO-criticality job that has not completed when its deadline arrives is *terminated*, not permitted to run on and complete late:

$$\text{job } J_{i,q} \text{ of } \tau_i \in \tau^{\text{LO}} \text{ released at } r \text{ is discarded at } r + D_i \text{ if incomplete.}$$

This is not a weakening of the timing guarantees; it is what makes them meaningful. A late result from a control or monitoring task may be worse than no result at all – it is delivered into a system state it no longer describes. Terminating is exactly the action two-level AMC already takes when it abandons LO-criticality jobs on a mode change; only the moment differs, and the moment is chosen so that the job never produces output it should not.

Three consequences follow, and Section 8.4 builds on all three.

1. **A LO-criticality deadline miss becomes impossible by construction.** There is nothing left to certify about a retained LO-criticality task’s deadline, which is why clause 2 of Section 7 is an objective rather than a constraint.
2. **HI-criticality safety is untouched, and if anything improves.** A terminated job executes no more than it otherwise would, so the interference a HI-criticality task suffers can only fall. Clause 1 is unchanged.
3. **Processor time can now be wasted.** A terminated job consumed execution that produced nothing. Abandonment on release never does this – it never abandons a job *mid-execution* – so WastedCPU was previously zero by construction and is now a live quantity.

8.4 Objective: Jobs Not Completed

The quantity to minimise is the work the system failed to deliver, however it failed. Let Exp be the number of LO-criticality jobs a perfect scheduler would have completed over a run of length H – a function of H and the periods alone, and so independent of anything the scheme did:

$$\text{Exp} = \sum_{\tau_i \in \tau^{LO}} |\{r = \phi_i + mT_i : m \in \mathbb{N}, r < H\}|.$$

Let Comp be the jobs that actually completed within their deadlines. Then

$$\text{JNC} = \text{Exp} - \text{Comp}, \quad \text{ServiceRatio} = \frac{\text{Comp}}{\text{Exp}}.$$

Defining JNC by *difference* against a fixed reference, rather than by summing the ways a job can fail, is deliberate: it cannot silently omit a failure mode. A job delivers no result by being abandoned on release (JNE), or by being terminated at its deadline having run – and JNE alone, the metric the two-level literature reports, counts only the first. It therefore *undercounts* once retained tasks are allowed to fail, which is precisely the regime this model introduces.

All metrics are reported normalised. A raw count of abandoned jobs scales with both the run length and the tasks’ periods, so raw JNE is not comparable between task sets or between experiments of different duration. Every rate below is a percentage of Exp ; WastedCPU is a percentage of H .

9 Two Operating Points

Clause 2 of Section 7 is a dial, not a decision, and the two ends of it are different schemes with different prerequisites. Both are reported, because they suit different certification appetites.

Point A – conservative. Clause 2 enforced. Every *retained* LO-criticality task is certified to complete, so LDM is zero by analysis for everything the scheme promises to keep. **This needs no new execution model:** it is sound under the original two-level semantics, and can be adopted by anyone unwilling to accept a task being cut off mid-computation.

Point B – termination. Clause 2 as an objective. A retained LO-criticality task is best-effort and is terminated at its deadline if it cannot finish (Section 8.3). It sheds markedly less, and requires accepting that a job may be discarded having run.

A third, intermediate policy sheds Point A’s set but defers the difference to rung 2, so tasks Point A abandons immediately instead run at L_1 . It sits between the two throughout, confirming the dial is continuous rather than a choice of two.

No HI-criticality deadline was missed under any policy in any regime, as clause 1 requires. Four things follow.

Both points beat two-level AMC, and the ordering never inverts. In this pilot, Point A gains 1.6 to 3.0 percentage points of service ratio and Point B a further 1.1 to 2.8 on top. The properly-powered regime map (mode_optimization.tex, “Stage Results”) confirms the ordering across a much wider grid – 32 cells spanning $U \in \{0.6, 0.7, 0.8, 0.9\}$, both deadline regimes, both policies – and finds larger gains throughout: +2.4% to +27.3%, every cell a win, widening as the regime hardens. The conservative point is worth having on its own regardless of the exact margin: it is a strict improvement over AMC-RH that costs no change to the execution model and gives up no guarantee AMC-RH provides.

Termination is insurance, not the source of the gain. Terminations never exceed 0.22% of jobs and wasted processor time never 0.32% of the run. Point B does not win by discarding late work – it barely discards any. It wins by *shedding less up front*, which termination is what makes sound. Reporting the gain as a benefit of termination itself would misattribute it.

The claimed loss at $U = 0.85$, tight, does not reproduce. This pilot’s own 3-seed, ~ 25 -task-set sample reported Point B 0.25 percentage points *worse* than AMC-RA here, and an earlier draft of this section concluded the scheme should not be recommended unconditionally on that basis. It should not

Table 5: Service ratio by operating point: an early, small pilot ($k = 4$, trigger severities $(0, 0.5, 1)$, $x_{LO} = 0$, $f_p = 0.2$, 200,000 ticks $\times$ 3 seeds, up to 30 task sets per block). **Superseded by the properly-powered study in mode_optimization.tex, “Stage Results”** – retained here because the correction it prompted (below) is itself part of the record. *Tight* deadlines sit 30% of the way from R_i^{lo} to T_i .

Regime	Policy	Service	JNE%	Term.%	Waste%
$U = 0.70$, implicit	Two-level AMC-RA	0.9390	–	–	–
	A: conservative	0.9549	4.48	0.00	0.00
	intermediate	0.9662	3.34	0.00	0.00
	B: termination	0.9728	2.69	0.00	0.00
$U = 0.70$, tight	Two-level AMC-RA	0.8551	–	–	–
	A: conservative	0.8767	12.12	0.19	0.13
	intermediate	0.8939	10.38	0.21	0.16
	B: termination	0.9050	9.26	0.22	0.18
$U = 0.85$, implicit	Two-level AMC-RA	0.8454	–	–	–
	A: conservative	0.8754	12.36	0.03	0.06
	intermediate	0.8839	11.51	0.04	0.07
	B: termination	0.8863	11.28	0.04	0.07
$U = 0.85$, tight	Two-level AMC-RA	0.6727	–	–	–
	A: conservative	0.6725	32.53	0.20	0.32
	intermediate	0.6703	32.74	0.20	0.32
	B: termination	0.6702	32.75	0.21	0.32

have: the sample’s own **resolvable** bound was never checked against this study’s 5% threshold before the claim was written down, and $U = 0.85/\text{tight}$ is a scarce region of the population (mode_optimization.tex, “Feasibility”), exactly where a small sample is least trustworthy. Re-measured independently at 50 task sets and 10 seeds: Point A scores +1.70% over AMC-RA (95% CI $[-0.10\%, +2.51\%]$, not resolved – the interval cannot rule out a very small loss, but overwhelmingly favours a small gain); Point B scores +5.87% (resolved, practically significant). The full regime map in mode_optimization.tex, “Stage Results”, finds the scheme beating AMC-RA in all 32 cells tested, including this one. The methodological lesson is the one this project’s own measurement contract already states: an unpowered pilot is not merely weaker evidence, it can be actively wrong, and this is a case of that happening in this project’s own paper rather than a hypothetical.

The intermediate policy is dominated but instructive. It never beats Point B, because the tasks it defers to rung 2 are precisely those the analysis could not certify – shedding them protects nobody else, since clause 1 and every certified task’s deadline already hold with the smaller set. Running them best-effort and terminating the failures is simply better than abandoning them on release. That is the clearest single argument for Point B.

10 Notation Summary: Task and Degradation Model

References are drawn first from the project-specific bibliography (project_bib.bib), then from the master Overleaf bibliography (references.bib).

11 Task Set Generation

Following DRS (RTSS 2020) [10], utilisation vectors are generated via the Dirichlet-Rescale algorithm.

Table 6: Notation Reference

Symbol	Meaning
τ	Task set
n	Number of tasks
k	Number of degradation levels ($k \geq 2$)
κ_i	Criticality of task i (HI or LO)
T_i, D_i	Period and deadline of task i
$C_i^{\text{lo}}, C_i^{\text{hi}}$	Normal and HI-criticality execution budgets
$\chi \in [0, 1]$	Severity; 0 = normal, 1 = fully degraded
$C_i(\chi)$	Execution budget for task i at severity χ
BCET_i	Best-case execution time
$U_i^{\text{lo}}, U_i^{\text{hi}}$	Utilisation in LO/HI modes
R_i^{lo}	Normal-mode response time (Section 5.1)
R_i^{hi}	Classic two-level HI-mode response time (Section 5.2)
$R_i(\chi)$	Severity-indexed response time / run-time trigger threshold (Section 5.3)
R_{trigger}	$R_i(\chi_{k-1})$, the deepest level's trigger
χ_x	Severity assigned to level L_x
S_x	Drop set at level L_x (Section 7)
ℓ_i	Allocated criticality level of task i (Section 4.3)
$R_i(\chi, S, s)$	Shed-aware response time (Section 5.4)
x_{LO}	Deepest rung a LO-criticality task may fire (Section 6.2)
Exp	LO jobs a perfect scheduler would complete (Section 8.4)
JNC	Jobs Not Completed, $\text{Exp} - \text{Comp}$ (Section 8.4)
ServiceRatio	Comp/Exp (Section 8.4)
$FP = 1/N$	HI-behaviour probability per job
$\tau^{\text{HI}}, \tau^{\text{LO}}$	HI/LO criticality subsets
L_x	Degradation level x ($0 \leq x < k$)

11.1 DRS Algorithm

Given n tasks and target utilisation U , the DRS algorithm produces $\mathbf{u} = (u_1, \dots, u_n)$ such that:

$$\sum_{i=1}^n u_i = U, \quad 0 \leq u_i \leq 1$$

The algorithm operates via:

1. Draw $\mathbf{x} \sim \text{Dirichlet}(1, \dots, 1)$ (uniform on simplex)
2. Scale: $u_i = U \cdot x_i$
3. Apply constraints via rescaling to standard simplex

11.2 Task Parameter Generation

For each task τ_i , parameters are generated as follows:

$$\begin{aligned}
T_i &\sim \text{LogUniform}(\log(T_{\min}), \log(T_{\max})) \\
D_i &= T_i \\
C_i^{(0)} &= \lfloor \lceil u_i \cdot T_i \rceil \rfloor \\
\text{BCET}_i &\sim \text{Uniform}(\text{BCET}_{\min} \cdot C_i^{(0)}, C_i^{(0)}) \\
C_i^{(k-1)} &= \min(\lfloor \lceil CF \cdot C_i^{(0)} \rceil \rfloor, T_i)
\end{aligned}$$

where:

- $T_{\min}$ and $T_{\max}$ are the minimum and maximum period bounds
- $BCET_{\min}$ is the minimum BCET fraction (typically 0.8)
- CF is the criticality factor relating normal to degraded budgets

12 Metrics

12.1 Degradation Metrics

NiD	Number of times degraded mode is entered (from L_0 only)
TiD	Fraction of time spent in degraded mode
JNE	Jobs Not Executed (LO jobs dropped)
Term.	Terminations (LO jobs discarded at their deadline, incomplete)
JNC	Jobs Not Completed = JNE + Term.; reported normalised
HDM	HI Deadline Miss (HI jobs missing deadline; must be zero for a safe scheme)
LevelTrans	Number of degradation-level changes, any direction, at any depth

LevelTrans generalises NiD for $k > 2$: NiD only counts transitions out of L_0 , so it cannot see oscillation confined to the degraded levels ($L_2 \leftrightarrow L_3$, say), which a badly-spaced severity ladder can produce. At $k = 2$ the two are proportional.

12.2 Objective Function

For k -level scheduling, we define the objective function:

$$\Phi = \alpha \cdot \mathbb{E}[\text{JNE}] + \beta \cdot \mathbb{E}[\text{TiD}] + \gamma \cdot \mathbb{E}[\text{WastedCPU}] + \delta \cdot \mathbb{E}[\text{LevelTrans}]$$

where:

- JNE: Jobs Not Executed (LO jobs dropped)
- TiD: Fraction of time spent in degraded mode
- WastedCPU: CPU cycles spent on abandoned LO work
- LevelTrans: Number of level changes (penalizes frequent or oscillating mode changes; see above)

NiD remains a reported metric for direct comparison against the two-level AMC-RH baseline, which only ever defines it; it is not double-counted in Φ once LevelTrans is included. The coefficients $\alpha, \beta, \gamma, \delta$ are weighting parameters that can be tuned based on application requirements.

REVIEW – This definition of Φ uses JNE. Later work (Section 6.2’s deadline-termination discussion; docs/metrics_objective.md) argues JNC = JNE + Term. is the primary metric instead, specifically because JNE alone is blind to jobs lost by termination rather than abandonment – and every Φ actually reported in this document’s results sections uses JNC-based service ratio, not this JNE-based formula. This section was not updated when that argument was made elsewhere. Found while assembling this report, not previously flagged: worth deciding whether to update this definition to JNC (matching practice) or state explicitly why JNE is the right choice here specifically.

13 Simulation Parameters

Common parameter values used in evaluation. These are the defaults of `amc_tasksim.generation.taskset.generate_tasks` quoted here as named values rather than literals so the two cannot silently drift apart – `tests/generation/test_paper_defaults` checks this table against the function’s actual defaults on every test run.

Table 7: Default Simulation Parameters

Parameter	Default Value	Description
n	20	Number of tasks
CP	0.5	Criticality proportion (fraction of tasks HI)
U	0.8	Target total LO-criticality utilisation
N	10000	Inverse HI-behaviour probability ($FP = 1/N$)
$T_{\min}$	100	Minimum task period (ticks)
$T_{\max}$	10000	Maximum task period (ticks)
$BCET_{\min}$	0.8	Minimum BCET as fraction of C_i^{lo}
CF	varies by experiment	Criticality factor – swept, not defaulted
k	2 (baseline)	Number of degradation levels – swept in $\{2, 3, 4\}$ per <code>amc_tasksim.experiments.com</code>

14 Notation Summary: Evaluation Metrics

Table 8: Notation Reference

Symbol	Meaning
n	Number of tasks
U	Target utilisation
$\mathbf{u}$	Utilisation vector
T_i	Period of task i
D_i	Deadline of task i
$C_i^{(0)}$	Normal execution budget
$C_i^{(k-1)}$	HI execution budget
$BCET_i$	Best-case execution time
$U_i^{\text{lo}}, U_i^{\text{hi}}$	Utilisation in LO/HI modes
$FP = 1/N$	HI-behaviour probability per job
CF	Criticality factor
k	Number of degradation levels
$\tau^{\text{HI}}, \tau^{\text{LO}}$	HI/LO criticality subsets

15 Optimisation Problem Statement

With the ladder mechanism and its safety argument established (Sections 2, 6.2 and 7) and the metrics and objective Φ defined (Section 12.2), one design question remains open: for k degradation levels, what severities $\chi_2, \dots, \chi_{k-1}$ should trigger levels $L_2, \dots, L_{k-1}$, and what method should be used to find them? The method cannot be chosen from habit or convenience – the wrong choice either wastes evaluation budget the problem does not need spent, or under-samples a domain that needs full coverage. This part characterises the search problem first (Section 16), then uses that characterisation to assess the candidate optimisation techniques (Section 17), states the resulting hypothesis (Section 18), and proposes the staged empirical protocol that tests it (Section 19) – itself already partly executed, not merely proposed, as the pilots in Section 19 show. A second question the protocol’s own findings raised but did not set out to answer – not just when to enter a degraded level but how gracefully to *leave* one – is addressed in Section 21.5.

16 Problem Characterisation: The Optimisation Landscape

16.1 Decision Variables and Domain

A k -level severity ladder has $k - 1$ trigger points, but the deepest, χ_{k-1} , is not free: the safety argument (admissible drop sets, `task_model.tex`) requires $\chi_1 = 0$, and pinning χ_{k-1} recovers the classic two-level guarantee exactly at the deepest level. The optimisation is therefore over $\mathbf{k} - 2$ free variables $\chi_2 \leq \chi_3 \leq \dots \leq \chi_{k-2}$, each bounded in $(0, 1]$ and subject to a total order. This is small in every case this work considers:

Table 9: Free dimensions and feasible-set size by level count

k	free dimensions	unordered hypercube ($g = 20$)	ordered feasible set
3	1	400	20
4	2	8,000	210
5	3	160,000	1,540

The right-hand column is not the hypercube g^{k-2} but the number of non-decreasing $(k-2)$ -tuples drawn from a grid of g values, $\binom{g+k-3}{k-2}$ (a standard multiset/”stars-and-bars” count): the ordering constraint is not an incidental detail but the dominant factor in how large the space actually is. A search method that samples the unconstrained box and repairs infeasible points by sorting – the natural approach for a population-based or surrogate-based method operating on a free vector – wastes evaluations on points that collapse onto the same feasible tuple, and in the worst case biases the sampling measure over the feasible region without the practitioner intending it to.

16.2 The Objective is a Noisy, Expensive-per-Sample Simulator Output

Φ (`metrics_objective.md`) is not observed directly; it is estimated from a discrete-event simulation whose execution times and HI-criticality occurrences are drawn at random. Per-seed coefficients of variation, measured on this simulator: 0.157 (NiD), 0.314 (JNE), 0.199 (degraded ticks). Neither the objective nor its gradient is available in closed form, so the landscape is a black box in the sense every method below has to contend with, not merely difficult to solve analytically.

A single simulation is cheap (≈ 43 ms for 10^6 ticks) and embarrassingly parallel across seeds and task sets, but the noise above means one evaluation at one lattice point is not one sample – it is a paired mean over a seed block, and the block has to be large enough to resolve the effect size the study cares about (§17 returns to what this implies for methods that spend their budget on *many* points rather than *repeated* samples at few).

16.3 What Is Not Yet Known

Whether Φ is unimodal, smooth, or has a plateau over the severity domain is an open empirical question, not an assumption available to lean on. A method whose efficiency depends on such structure – surrogate modelling, gradient estimation, population convergence – is therefore choosing to bet on a property this document cannot yet certify. A method that does not depend on it is not making that bet, at the cost of not exploiting the structure if it does turn out to hold. This tension is the crux of Section 17, and Section 19 answers it empirically rather than leaving it open.

17 Related Work: Optimisation Techniques for Expensive Stochastic Objectives

Optimising the parameters of a stochastic simulation, rather than a closed-form function, is the subject of *simulation optimisation*, whose central tension Fu [7] frames precisely: theoretical convergence results are

proven for algorithms that are rarely what practitioners actually run, while the general-purpose metaheuristics that are widely used borrow convergence properties from deterministic optimisation that do not transfer cleanly to a noisy objective. Four families of technique are relevant to the problem in Section 16.

17.1 Exhaustive Search and Design of Experiments

Design of Experiments (DOE) fits a statistical model – typically linear or low-order polynomial – to a systematically chosen sample of the parameter space, trading exhaustive coverage for a smaller, structured design (e.g. a factorial or Latin-hypercube design) when the space is too large to enumerate. Tate et al. [19] apply exactly this trade-off to a directly comparable problem: tuning multiple sensornet protocol parameters against several competing performance metrics, a genuinely high-dimensional, combinatorially explosive space where full enumeration is not an option. Their DOE fit yielded a more generally applicable model of the parameter space; a competing evolutionary search (Section 17.2) found a broader range of viable solutions at lower cost for the specific instance evaluated. That finding does not transfer here, and the reason it does not is itself informative: their conclusion favours evolutionary search precisely where full enumeration is infeasible, and Section 16 establishes that the severity ladder is not in that regime – 1,540 points at $k = 5$ is exhaustively enumerable, so DOE’s usual purpose, choosing which subset of a space to sample because the whole space cannot be, does not arise. What remains applicable is the weaker, unconditional claim: over a space small enough to enumerate, exhaustive search dominates any sampling design of it, since a sample is only ever a strict subset of what enumeration already covers, at correspondingly less information per unit of the region actually examined.

17.2 Evolutionary and Population-Based Metaheuristics

Genetic algorithms [8, 21] and related population-based metaheuristics such as simulated annealing [12] search by iteratively perturbing and selecting among a working set of candidate solutions, and are the default choice when a space is too large or too irregular to enumerate and no exploitable structure is known in advance – exactly the case in Tate et al.’s protocol-tuning problem above. Applied here, the same population-based iteration that gives these methods their reach becomes a liability: each generation adds evaluations, and Section 16’s per-seed noise means every added evaluation is a further noisy draw. Selecting the best of M such draws is optimistically biased by $\sigma\sqrt{2\ln M}$ (a standard result for the expected maximum of M noisy samples). A population of 50 run for 100 generations – an unremarkable configuration – makes 5,000 evaluations; at this simulator’s measured noise, that inflates the reported optimum’s bias to roughly 12%, more than double the 5% practical-significance threshold this work uses to call a result meaningful (metrics_objective.md). *A genetic algorithm run against a perfectly flat objective would return a publishable-looking optimum.* This is a general property of any method whose evaluation count scales with iterations rather than with the size of the space being covered, and it argues against every method in this family for a domain this small, independent of how well-suited any of them are in a domain too large to enumerate.

17.3 Bayesian Optimisation and Surrogate Models

Bayesian optimisation [11, 17] fits a probabilistic surrogate – typically a Gaussian process – to the evaluations seen so far, and chooses the next point by an acquisition function that balances exploiting the surrogate’s optimum against exploring where it is most uncertain. It is the standard choice when each evaluation is expensive enough that minimising the *number* of evaluations dominates every other concern, and its sample efficiency is real precisely under that condition. It is not the condition here: Section 16 costs a single simulation at tens of milliseconds and the whole lattice at a few CPU-hours, so there is no evaluation budget for a surrogate to conserve. Two further mismatches follow from Section 16 specifically. First, the deliverable is a response surface, not an optimum – Bayesian optimisation is built to concentrate samples near a believed optimum, which is the opposite of what characterising a surface requires, and recovering the surface afterwards needs a further, separately budgeted sampling pass. Second, the surrogate assumes a smoothness the objective has not yet been shown to have (Section 16); a Gaussian-process surrogate

additionally needs an explicit noise term fitted correctly to the measured heteroscedastic variance above, which is one more unverified assumption riding on an already-unverified one.

17.4 Gradient-Based and Derivative-Free Local Search

Classical gradient-based optimisation is inapplicable outright: Φ is the output of a discrete-event simulation with no closed form and no analytic derivative, and finite-difference gradient estimates would themselves be noisy at the measured coefficients of variation, compounding rather than resolving the problem Section 17.2 raises. Derivative-free local search (pattern search, Nelder–Mead) removes the need for a derivative but keeps the core liability: it is iterative and local, so it inherits both the winner’s curse of repeated noisy evaluation and a dependence on the smoothness this document has not established.

18 Working Hypothesis, and Why It Is Not Yet a Conclusion

Exhaustive enumeration of the ordered severity lattice (Section 16), evaluated under common random numbers. Every technique surveyed in Section 17 earns its place by trading coverage for sample efficiency in a large or unstructured space; the severity ladder’s space is neither. DOE’s usual justification – covering a space too large to enumerate – does not apply once the ordering constraint is respected directly rather than repaired after the fact. Evolutionary and Bayesian methods both spend evaluations to save evaluations, at a specific, quantified cost (winner’s-curse bias, or an unrepresentative sample of the surface) that this domain does not need paid. Enumeration additionally reports the whole surface natively, has one hyperparameter – grid resolution – against a genetic algorithm’s at least eight or a Gaussian process’s comparable number, and needs no additional dependency.

That argument is structural: it follows from counting the feasible set and from the measured cost of one evaluation, and it does not depend on knowing Φ ’s shape. It is not, on its own, an empirical result. It has not been checked against a real comparison between methods, and the pilot in Section 19 shows at least one place a plausible intuition about this problem turned out to be wrong when actually measured (Section 21.9) – which is reason enough not to extend the same trust to an argument that has not been tested at all. Section 19 is the protocol that tests it, structured so that if the hypothesis is wrong, that becomes visible before it is load-bearing for the rest of this work, rather than after.

Whichever method the protocol below settles on, it must respect the noise floor Section 16 establishes: configurations are compared as paired samples under common random numbers, never as independent means, since an unpaired comparison at this simulator’s measured noise needs roughly two orders of magnitude more task sets to resolve the same effect (`metrics_objective.md`, `evaluation_methodology.tex` §3). This is the one requirement every method in Section 17 shares, and the one a choice of *method* cannot substitute for.

Retrospective note. The hypothesis is confirmed, but not by the comparison this section anticipated running. Section 21.3 shows exhaustive search’s own upper bound on what any method could win over the default configuration is 0.79% – so the reason enumeration was the right choice is not that it beat a genetic algorithm in a head-to-head test, but that the prize on offer was never large enough for the choice of method to matter. That is a stronger vindication of the structural argument above than a head-to-head win would have been: it is not that enumeration happened to find a better point, it is that no method could have found a materially better one.

19 A Staged Empirical Protocol

Choosing a method well needs three things a one-shot argument cannot supply: a measured landscape to reason from instead of an assumed one, a real comparison between candidate methods rather than a critique of each in isolation, and a evaluation budget sized to the noise actually observed rather than picked by convention. The protocol below produces each in turn, each phase’s output gating the next. Two of the three phases already have a worked pilot with real data, run on the infrastructure that exists today (Section 16’s severity-indexed response time and the two-level engine); the third needs the k -level engine that does not yet exist and is specified precisely so it is ready once that engine does.

20 Revised Protocol

The protocol in Sections 21.7 to 21.9 was designed before the k -level engine existed, and assumed that trigger spacing was the quantity worth optimising. With the engine built and verified, a structural pilot contradicts that assumption outright, and the protocol has to be rebuilt around what the pilot found rather than around what was expected.

20.1 What the structural pilot found

All figures below are paired (identical task sets and seed blocks), aggregated one value per task set, and reported against this study’s declared 5% practical-significance threshold. $n = 12$, $U = 0.7$, $CF = 2$, $f_p = 0.2$, 200,000 ticks.

Table 10: Effect of each design knob on service ratio

Knob	Effect	Resolvable at	Verdict
Scheme vs. two-level AMC	+5.5 to +11.9%	—	above threshold
Progressive vs. shed-early	+0.24 to +0.62%	0.25–0.79%	1 of 4 resolved
Shedding order	+0.96 to +1.54%	1.2–1.4%	1 of 2 resolved
Greedy vs. exhaustive optimum	+0.29 to +0.83%	0.3–1.0%	not resolved
Severity spacing (shed-early)	exactly 0	—	structurally vacuous
$k \in \{2, 3, 4, 5\}$ (shed-early)	exactly 0	—	structurally vacuous
x_{LO}	≈ 0 , slightly negative	—	settled negative

Two entries are exact zeros rather than small numbers, and the reason is structural rather than statistical. `drop_set_shed_early` requires one drop set feasible at *every* operating severity, and the deepest is pinned to 1; by monotonicity (M1) feasibility there implies it everywhere below, so the intermediate severities never bind and the drop set is identical for every spacing – confirmed on 16/16 task sets. Since operating budgets are an analysis quantity and not a run-time throttle (see the task model’s “Execution Budget Enforcement”), the drop set is the *only* thing that distinguishes one level from another. Identical drop sets therefore make the levels observationally identical, and k has no effect: service ratio agrees to six decimal places across $k \in \{2, 3, 4, 5\}$.

Consequence. Under shed-early the k -level ladder collapses to two levels, and the entire gain over two-level AMC comes from *shedding a subset rather than everything* – not from grading. Grading only becomes observable under a *progressive* ladder, where $S_1 \subsetneq S_{k-1}$, and there it is real: a progressive ladder at severities (0, 0.25) sheds 14.6% at the shallowest rung under the conservative point and 0.0% under the termination point, against shed-early’s 69.4% and 57.6%. That is the original promise of graded degradation – mild overruns needing no shedding at all – and it is the one configuration in which the multi-level structure earns its keep.

The cost is feasibility, and it is the headline risk. A progressive ladder does not always exist: 16/24 task sets at severities (0, 0.25), and only 6–7/24 at (0, 0.5), because a task first shed at a deep rung has an infinite shed instant for 33% of HI-criticality tasks at $\chi = 1$. Shed-early exists always. Applicability, not optimality, is therefore the quantity the study should be measuring.

20.2 Why the original Phase B is retired

Phase B asked which optimisation method finds the best trigger spacing under a matched budget. Trigger spacing has *no effect at all* under the policy the carry-in analysis endorses, so the question is vacuous there; and where it is not vacuous, every knob in Table 10 moves the objective by under 1.6%, against a declared threshold of 5%. The whole answer space of the method comparison lies an order of magnitude below what this study calls a meaningful difference. Running a genetic algorithm against enumeration to compete for tenths of a percent would produce a number, not a finding.

The exhaustive optimum is itself only 0.29–0.83% better than a greedy shedding order, and that gap is not resolvable at the pilot’s scale. This retires the metaheuristic question *with evidence* rather than by

assertion, which is what Phase B existed to do – it simply answers it in the negative and far more cheaply than the original design would have.

20.3 The replacement: four stages

Stage 0 – structural pruning. Before measuring anything, establish which knobs can have an effect. Costs minutes, and it is what revealed the two exact zeros above. Must be re-run whenever the execution model changes, because both zeros are consequences of the model rather than of the task sets.

Stage 1 – feasibility characterisation. For each configuration (operating point $\times$ policy $\times$ severity vector), what fraction of task sets admit a ladder at all? This is a design-time property needing no simulation, so it is cheap, and it is the primary applicability result: a configuration that wins by 6% on a quarter of task sets is a different proposition from one that wins by 0.6% on all of them.

Stage 2 – regime map. The main experiment, and the only one whose effects exceed threshold. Service ratio against two-level AMC across $(U, D_i/T_i, CF, f_p)$, for both operating points, reporting where the scheme wins, by how much, and *where it loses* – at $U = 0.85$ with tight deadlines it already loses by 0.25 percentage points, and that boundary is a result, not an embarrassment.

Stage 3 – configuration selection under a bounded-gain budget. Since every knob is sub-threshold, the question is not “find the optimum” but “is the simplest configuration good enough, and by how much could anything beat it?” Deliverable: an upper bound on what any search could win, obtained by exhaustive enumeration on small task sets. If that bound is below threshold – as the pilot suggests – the simplest configuration is selected and the matter is closed.

Stage 4 – budget calibration, per operating point. The two points do not behave alike and must be sized separately: the shedding-order effect is resolved under the conservative point (+1.54% against 1.42% resolvable) and not under the termination point (+0.96% against 1.17%). Sizing one study from the other’s pilot would under-power it.

21 Stage Results

Every number below is produced by `amc_tasksim.experiments.multilevel_protocol`, tested in `tests/experiments/test`, and reproducible from the module’s public functions – this section is a report of what running them found, not a hand-computed table.

21.1 Stage 1: Feasibility

Forty task sets per cell ($n = 10$, $CP = 0.5$, $CF = 2$), swept over utilisation, deadline tightness, operating point and severity gap. No simulation is required – feasibility is decided by response-time analysis alone.

Three findings, in order of how much they change the earlier picture.

Shed-early is feasible everywhere, always ($P = 1.00$). Expected: shedding everything at R_i^{lo} is AMC-rtb’s own equation, so shed-early is certifiable exactly when the task set is AMC-rtb-schedulable, which is the population’s qualification criterion.

Progressive feasibility falls as the regime hardens. From 90% ($U = 0.6$, implicit) to as low as 50% ($U = 0.8$, tight, point B) and the population itself grows scarce at $U = 0.9$ tight ($40 \rightarrow 12$ candidates found in the same search budget). This is the applicability cost Section 21.7’s successor was designed to surface: a progressive ladder is not a free upgrade over shed-early, it is a bet that may not be available when the regime is hardest – precisely where the gain would matter most.

Under Point B, the shallowest rung sheds nothing wherever it is feasible. `shed%(progressive)` is exactly 0.0% in every Point-B row at this spacing. This is not an averaging artefact: every task set that admits a progressive ladder at $(0, 0.25)$ under the termination point does so while shedding zero LO-criticality tasks at L_1 – full retention, right up until the deeper rung. Under Point A the same column ranges from 10.6% to 77.5%, because clause 2’s per-rung certification is the binding constraint and it tightens as U and deadline pressure rise.

Severity spacing is not vacuous for progressive ladders – but it is close to vacuous for *service*. Section 20.1 showed spacing has no effect at all under shed-early. Under a progressive ladder it is not vacuous in this narrower sense: L_1 ’s *operating* severity is `severities[1]`, so widening the first gap directly raises what

Table 11: Feasibility at the most permissive progressive spacing, $(0, 0.25)$. $P(\cdot)$ is the fraction of task sets admitting the ladder; shed% is the mean percentage of LO-criticality tasks shed at the shallowest rung, over task sets where the ladder exists.

U	Regime	Point	n	$P(\text{shed-early})$	$P(\text{progressive})$	shed%(progressive)
0.6	implicit	A	40	1.00	0.90	10.6%
0.6	implicit	B	40	1.00	0.90	0.0%
0.6	tight	A	40	1.00	0.72	43.4%
0.6	tight	B	40	1.00	0.68	0.0%
0.7	implicit	A	40	1.00	0.70	17.9%
0.7	implicit	B	40	1.00	0.65	0.0%
0.7	tight	A	40	1.00	0.65	46.2%
0.7	tight	B	40	1.00	0.60	0.0%
0.8	implicit	A	40	1.00	0.60	27.5%
0.8	implicit	B	40	1.00	0.57	0.0%
0.8	tight	A	40	1.00	0.57	64.3%
0.8	tight	B	40	1.00	0.50	0.0%
0.9	implicit	A	40	1.00	0.70	57.1%
0.9	implicit	B	40	1.00	0.65	0.0%
0.9	tight	A	12	1.00	0.67	77.5%
0.9	tight	B	12	1.00	0.58	0.0%

L_1 is charged and, correspondingly, how much it must shed to stay certified. At $U = 0.7$, implicit, point A: shed%(progressive) at L_1 rises from 17.9% at spacing $(0, 0.25)$ to 46.7% at $(0, 0.5)$ to 64.0% at $(0, 0.75)$, while feasibility falls from 70% to 38% to 38%. But a paired check of whether this moves *service ratio* (not just shed-set size), on the 30-task-set pilot restricted to sets feasible under both spacings compared, resolved in only 1 of 6 comparisons tested (point B, $(0, 0.25) \rightarrow (0, 0.5)$, +0.58%, $n = 8$) – everywhere else the difference was not distinguishable from zero at that sample size. Spacing controls what is *shed*, robustly; whether that translates into *delivered service* is a smaller and noisier effect, consistent with every other sub-threshold knob in Table 10.

21.2 Stage 2: The Regime Map

Thirty task sets per (U, regime) cell ($n = 10$, $CP = 0.5$, $CF = 2$, $f_p = 0.2$, 200,000 ticks, 8 seeds), paired against two-level AMC-RA computed on the same task sets and seeds. Progressive uses severities $(0, 0.25)$, the most feasible spacing found in Stage 1.

The scheme beats AMC-RA in every one of the 32 cells tested. Every single diff in Tables 12 and 13 is positive, and 22 of 32 are practically significant by this study’s own threshold. Service gains widen as the regime hardens: from +2.4% ($U = 0.6$, tight, point A, shed-early) up to +27.4% ($U = 0.9$, tight, point B, progressive). Progressive beats shed-early in every cell where both are feasible, and Point B beats Point A in every cell – both orderings are exactly what `task_model.tex`’s “Mode Transition Protocol” and “Shed-Aware Response Time” sections predict, now confirmed across the grid rather than on isolated pilots.

A methodological correction, reported because it happened. An earlier version of this analysis reported the scheme *losing* to AMC-RA at $U = 0.85$, tight deadlines, by 0.25 percentage points (0.6702 against 0.6727), computed from a 25–30-task-set, 3-seed pilot. That result does not reproduce. Re-measured independently with a fresh 50-task-set population at the same regime, 10 seeds, 300,000 ticks: Point A scores 0.7195 against AMC-RA’s 0.7074 (+1.70%, 95% CI $[-0.10\%, +2.51\%]$, not resolved – the interval cannot rule out a very small loss, but overwhelmingly favours a small gain over a loss); Point B scores 0.7489 (+5.87%, 95% CI $[+1.75\%, +6.55\%]$, resolved and practically significant). Neither result supports the original claim of an outright loss.

The likely cause is exactly what Section 21.9 (Stage 4’s predecessor) exists to prevent: $U = 0.85$ with tight deadlines sits in a scarce region of the population (Section 21.1), and a 3-seed, ~ 25 -task-set sample is not powered to distinguish a small effect from noise in a scarce, high-variance regime – the sample’s own

Table 12: Regime map, implicit deadlines ($D_i = T_i$). $n = 10$, $CP = 0.5$, $CF = 2$, $f_p = 0.2$, 200,000 ticks $\times$ 8 seeds. Progressive severities (0, 0.25). ‘Point’ is the operating point (A conservative, B termination); ‘Scheme’ is the scheme’s service ratio; ‘Diff’ and ‘Rel.’ are the paired difference from AMC-RA; ‘Resolvable’ is the smallest relative effect this sample could distinguish from zero; ‘Verdict’ follows contract.PairedResult.

U	Point	Policy	n	AMC-RA	Scheme	Diff	Rel.	Resolvable	Verdict
0.60	A	shed early	30	0.9426	0.9729	+0.0303	+3.21%	1.61%	sig.
0.60	A	progressive	28	0.9430	0.9790	+0.0361	+3.83%	1.64%	sig.
0.60	B	shed early	30	0.9426	0.9848	+0.0422	+4.47%	2.06%	sig.
0.60	B	progressive	28	0.9430	0.9890	+0.0461	+4.89%	2.10%	sig.
0.70	A	shed early	30	0.9074	0.9445	+0.0371	+4.09%	2.58%	sig.
0.70	A	progressive	20	0.8909	0.9532	+0.0623	+6.99%	3.60%	practsig
0.70	B	shed early	30	0.9074	0.9591	+0.0517	+5.69%	3.23%	practsig
0.70	B	progressive	18	0.8938	0.9776	+0.0838	+9.38%	4.82%	practsig
0.80	A	shed early	30	0.8678	0.9017	+0.0339	+3.91%	2.89%	sig.
0.80	A	progressive	20	0.8515	0.9246	+0.0731	+8.58%	4.12%	practsig
0.80	B	shed early	30	0.8678	0.9283	+0.0605	+6.97%	3.68%	practsig
0.80	B	progressive	19	0.8490	0.9520	+0.1030	+12.14%	4.98%	practsig
0.90	A	shed early	30	0.7401	0.7878	+0.0477	+6.44%	4.31%	practsig
0.90	A	progressive	22	0.7331	0.8211	+0.0881	+12.01%	5.04%	practsig
0.90	B	shed early	30	0.7401	0.8441	+0.1040	+14.05%	6.77%	practsig
0.90	B	progressive	20	0.7307	0.8936	+0.1629	+22.29%	8.19%	practsig

resolvable bound was never checked against EFFECT_FLOOR before the claim was written down. This is retained here, rather than silently corrected, as the concrete case for running Stage 4 before trusting a small pilot: the failure mode it warns against is not hypothetical, it happened in this project’s own paper.

Consequence for task_model.tex’s “Two Operating Points” section. The claim that “both points degenerate in the hardest regime tested” and that “neither point should be recommended unconditionally” should be read as superseded by this section. Across the full grid tested here, including the specific cell the original claim was about, no loss was found once the comparison was properly powered.

21.3 Stage 3: Bounded-Gain Configuration Selection

Exhaustive search over every LO-criticality subset, $n = 8$ tasks, 14 task sets, 6 seeds, 150,000 ticks – tractable only at this scale, which is the point: it bounds what *any* search could win over the default (execution-time-ordered shed-early) configuration.

The largest gain any exhaustive search found over the default, anywhere tested, is 0.79% – an order of magnitude below the 5% threshold, and this is an *upper bound*: nothing outperforms exhaustive enumeration by construction. This closes the question Phase B was built to answer: no optimisation method, however good, has more than 0.79 percentage points of service ratio available to find in this configuration space. The default configuration is adopted, and the matter is closed.

21.4 Stage 4: Budget Calibration

The regime-map effect needs no calibration – it is resolved at 30 task sets in 26 of 32 cells already. What needs sizing is the knob on the edge of resolution: shedding order. Two independent pilots of the identical comparison (by_execution_time against by_utilisation, $U = 0.7$, shed-early) illustrate why a single pilot cannot be trusted for this:

Point A’s two estimates agree reasonably (1.54% against 1.33%). Point B’s do not: 0.96% against 0.11%, an order of magnitude apart, from two population draws of the same size and the same generation parameters. Both are below EFFECT_FLOOR, so the disagreement changes no conclusion – but it is exactly the instability a

Table 13: Regime map, tight deadlines (D_i at 30% of the way from R_i^{lo} to T_i). Same configuration as Table 12. n shrinks at $U = 0.90$ because qualifying task sets at this combination of high utilisation and tight deadlines are scarce (Section 21.1).

U	Point	Policy	n	AMC-RA	Scheme	Diff	Rel.	Resolvable	Verdict
0.60	A	shed early	30	0.9046	0.9263	+0.0217	+2.40%	1.89%	sig.
0.60	A	progressive	21	0.8886	0.9319	+0.0433	+4.87%	2.54%	sig.
0.60	B	shed early	30	0.9046	0.9364	+0.0318	+3.51%	1.93%	sig.
0.60	B	progressive	19	0.8883	0.9488	+0.0605	+6.81%	2.59%	practsig
0.70	A	shed early	30	0.8231	0.8658	+0.0427	+5.19%	2.74%	practsig
0.70	A	progressive	21	0.8301	0.8951	+0.0650	+7.83%	3.30%	practsig
0.70	B	shed early	30	0.8231	0.8884	+0.0654	+7.94%	3.18%	practsig
0.70	B	progressive	19	0.8215	0.9246	+0.1032	+12.56%	3.78%	practsig
0.80	A	shed early	30	0.7542	0.7770	+0.0228	+3.03%	2.89%	sig.
0.80	A	progressive	18	0.7453	0.7897	+0.0443	+5.95%	3.35%	practsig
0.80	B	shed early	30	0.7542	0.8116	+0.0575	+7.62%	4.66%	practsig
0.80	B	progressive	16	0.7479	0.8648	+0.1169	+15.63%	6.04%	practsig
0.90	A	shed early	11	0.6555	0.7208	+0.0653	+9.96%	8.51%	practsig
0.90	A	progressive	8	0.6758	0.7596	+0.0839	+12.41%	9.64%	practsig
0.90	B	shed early	11	0.6555	0.7889	+0.1335	+20.37%	10.37%	practsig
0.90	B	progressive	7	0.6788	0.8644	+0.1856	+27.35%	10.66%	practsig

Table 14: Exhaustive optimum against the default configuration.

Point	U	Default shed	Best shed	Gain	Verdict
A	0.70	2.8 tasks	2.6 tasks	+0.01%	not resolved
A	0.85	3.3 tasks	3.3 tasks	+0.00%	not resolved
B	0.70	2.6 tasks	2.4 tasks	+0.02%	resolved (negligible)
B	0.85	2.6 tasks	2.1 tasks	+0.79%	resolved (negligible)

single small pilot cannot be trusted to report honestly, and it is the same failure mode behind the $U = 0.85$ correction in Section 21.2. `required_pairs()` applied to Pilot 2 asks for as few as 1–11 task sets to resolve *its own* observed effect at the 5% floor – which is not reassuring, since Pilot 2’s effect is itself the unstable number in question. The honest conclusion is not “run a bigger ordering study”; it is the one Section 21.3 already reached by a different route: the ordering effect is too small to matter, and further calibration would only be calibrating noise.

21.5 Stage 5: Evidence-Cleared Exit

Stages 0–4 fixed *entry*: which severities to use, how many levels, which tasks to shed, in what order – and closed every one of those questions, because every knob moved the objective by under 1.6% (Section 22). None of them touched *exit*. The shipped scheme leaves a degraded level only on an idle instant, generalising AMC-RA; task_model.tex’s “Mode Transition Protocol” states the exit rule as “direct or cascade, per protocol design” and leaves it open. This section is a fifth stage, added after the four above concluded, asking the question they did not: does exit timing matter as much as entry configuration turned out not to?

A confound, found before trusting the headline number. The first cut of a diagnostic measuring time spent degraded beyond what active evidence justifies returned 62–91% across the grid – implausibly large next to every other effect in this study. Tracing a real run explained why: the HI-criticality job that triggers entry typically completes quickly, but the run-queue backlog it leaves behind takes much longer to drain, and the system correctly stays degraded until that backlog clears. A busy queue draining after

Table 15: The same comparison, two independent 24-task-set pilots.

Point	Pilot 1 effect	Pilot 2 effect	Task sets needed (Pilot 2’s estimate)
A	+1.54%	+1.33%	11
B	+0.96%	+0.11%	1

its trigger has gone is not a cost. The metric used below instead – LO-criticality jobs actually abandoned on release *during* that unjustified tail – is not subject to this confound: a job dropped during the tail is a countable instance of exactly the service loss a smarter exit rule could plausibly have avoided.

The reframe that made this tractable. `shed_early` (the adopted default) uses a single drop set at every level ≥ 1 – there is no intermediate level to cascade *through*, so for this policy the only lever an exit rule has is *when* the system returns to L_0 , not *which* drop set applies along the way. That is exactly the question AMC-RH already answers, with a rule that already exists, is already tested, and already has a published safety argument: exit to L_0 the instant no active, eligible job’s own threshold remains met, rather than waiting for the queue to idle. `simulation.multilevel` now implements this as `exit_policy="amc_rh"`, verified bit-for-bit against `engine.py`’s `AMC_RH` at $k = 2$ (`test_k2_full_drop_amc_rh_exit_reproduces_amc_rh_exactly`), and proven safe – scoped explicitly to full exit only, not partial demotion – as Corollary 2 of the safety proof.

REVIEW – Corollary 2 (`safety_proof.md`) rests on $\text{natural}(t)=0$ being the *exact* safe-to-exit boundary, not an approximation of it – the argument is that $R_i(\chi)$ is a worst-case bound under full interference, so a task that hasn’t crossed it is already safe under full interference regardless of what happens next. Please check that reasoning independently: it is new this pass, produced by the same process that also produced (and had to retract) the whole-run TiD overclaim flagged further down. If it holds, everything downstream in this section inherits from it.

Table 16: Service ratio, `exit_policy="amc_rh"` (candidate) paired against "idle" (baseline, today’s shipped behaviour), `shed_early` only, both operating points. $n = 24$ task sets ($n = 8$ at $U = 0.90/\text{tight}$ – population scarcity, Section 21.1), 5 seeds, 200,000 ticks, $f_p = 0.2$.

U	Regime	Point	n	Idle	AMC-RH-exit	Diff	Rel.	Verdict
0.60	implicit	A	24	0.9747	0.9942	+0.0195	+2.00%	sig.
0.60	implicit	B	24	0.9837	0.9957	+0.0120	+1.22%	sig.
0.60	tight	A	24	0.9259	0.9802	+0.0543	+5.86%	practsig
0.60	tight	B	24	0.9373	0.9825	+0.0451	+4.81%	sig.
0.70	implicit	A	24	0.9420	0.9832	+0.0411	+4.37%	sig.
0.70	implicit	B	24	0.9601	0.9868	+0.0267	+2.78%	sig.
0.70	tight	A	24	0.8566	0.9654	+0.1088	+12.70%	practsig
0.70	tight	B	24	0.8711	0.9688	+0.0976	+11.21%	practsig
0.80	implicit	A	24	0.9057	0.9779	+0.0723	+7.98%	practsig
0.80	implicit	B	24	0.9289	0.9813	+0.0524	+5.65%	practsig
0.80	tight	A	24	0.7772	0.9502	+0.1731	+22.27%	practsig
0.80	tight	B	24	0.8070	0.9543	+0.1473	+18.25%	practsig
0.90	implicit	A	24	0.8118	0.9610	+0.1492	+18.37%	practsig
0.90	implicit	B	24	0.8471	0.9648	+0.1177	+13.89%	practsig
0.90	tight	A	8	0.7355	0.9332	+0.1977	+26.89%	practsig
0.90	tight	B	8	0.7948	0.9418	+0.1469	+18.49%	practsig

The benefit is real, resolved, and the largest exit-related effect this study has found. Every

one of the 16 cells is a positive difference; 11 are practically significant, growing with utilisation and with tight deadlines, from +2.0% at $U = 0.6$ to +22–27% at $U = 0.8$ – 0.9 . That is an order of magnitude above every entry-side knob this study closed (Section 22: $\leq 1.6\%$) and above Stage 3’s exhaustive bound on drop-strategy search ($\leq 0.79\%$).

Table 17: Level transitions per run – the oscillation cost, tracked deliberately rather than assumed away. Same cells, configuration and pairing as Table 16.

U	Regime	Point	Idle	AMC-RH-exit	Diff	Rel.	Verdict
0.60	implicit	A	129.1	161.9	+32.8	+25.4%	practsig
0.60	implicit	B	125.6	161.9	+36.3	+28.9%	practsig
0.60	tight	A	323.8	390.1	+66.3	+20.5%	practsig
0.60	tight	B	322.6	390.0	+67.5	+20.9%	practsig
0.70	implicit	A	166.3	222.3	+55.9	+33.6%	practsig
0.70	implicit	B	156.5	222.0	+65.5	+41.9%	practsig
0.70	tight	A	402.2	522.4	+120.2	+29.9%	practsig
0.70	tight	B	398.4	522.6	+124.2	+31.2%	practsig
0.80	implicit	A	147.8	209.6	+61.8	+41.8%	practsig
0.80	implicit	B	140.8	209.7	+68.8	+48.9%	practsig
0.80	tight	A	396.7	568.3	+171.6	+43.3%	practsig
0.80	tight	B	380.8	568.1	+187.3	+49.2%	practsig
0.90	implicit	A	185.6	300.4	+114.7	+61.8%	practsig
0.90	implicit	B	160.7	300.4	+139.7	+86.9%	practsig
0.90	tight	A	411.4	631.6	+220.2	+53.5%	practsig
0.90	tight	B	356.4	631.6	+275.2	+77.2%	practsig

The oscillation cost is equally real, and it is not a rounding artefact. Every one of the 16 cells shows a practically significant *increase* in level transitions, 20–87%, on baselines already in the hundreds per run (125–411), so the increase is substantial in absolute count too (+33 to +275 transitions), not merely a large percentage of a small number. It grows with utilisation in lockstep with the service-ratio benefit – exactly the same cells that gain the most also oscillate the most. This is the direct, measured answer to whether evidence-cleared exit trades service for churn: it does, and the trade is not free in either direction. Whether that trade is worth taking is an editorial judgement this document does not make unilaterally; δ , the fixed weight metrics_objective.md assigns level transitions in Φ , is deliberately small relative to $\alpha(U)$ and $\beta(U)$, which is this project’s own prior that churn matters less than service loss and time degraded – but a 20–87% relative increase is large enough that this prior should be checked against the combined objective explicitly before the result is presented as an unqualified win, not assumed from the weight alone.

DECISION – No implementation of Φ exists in the codebase to check this against (Stages 1–4 reported service ratio directly, never a combined score). Two ways forward: (a) implement Φ with metrics_objective.md’s adaptive weights and re-score every exit-policy result already measured, or (b) skip Φ and make an explicit engineering call in prose (“we accept up to X% more transitions for Y% service gain”). (a) is more defensible but is new implementation work with its own validation burden; (b) is faster but is a judgement call this document has so far declined to make unilaterally. Which do you want?

21.5.1 Tempering the churn: a hold-off, and where its sweet spot moves

The oscillation cost above motivates a tempered exit: leave a degraded level once evidence has been continuously clear for `hold_off` ticks, rather than immediately. Unlike evidence-cleared exit, a hold-off’s deadline is a *timed* condition, not an event-triggered one, so realising it needed new machinery – an extension of the existing next-event lookahead (`_next_evidence_reappearance`) so the simulation loop cannot skip past evidence reappearing between otherwise-scheduled events. Two limits make `exit_policy="hysteresis"` an exact interpolation between the two policies already measured, not merely a variant of them: `hold_off=0` reproduces `exit_policy="amc_rh"` bit-for-bit, and a `hold_off` longer than any realistic gap reproduces `exit_policy="idle"` bit-for-bit – both verified in `tests/simulation/test_multilevel.py`, not argued.

Safety does not depend on hold_off, or on the lookahead’s precision. $R_i(\chi)$ is a worst-case bound computed under full interference, so a task that has not yet crossed it is guaranteed safe under full interference from that instant on, regardless of what happens next – $\text{natural}(t) = 0$ is not a reasonable proxy for the safe-to-exit boundary, it *is* that boundary, exactly. Every exit under `"hysteresis"` is gated on a *fresh* check of this condition at the instant it fires, never on the hold-off bookkeeping alone, so an imprecise or delayed wake-up can only delay an individually-safe exit, never bring one forward into unsafe territory. This is Corollary 3 of the safety proof (`safety_proof.md`); it needs nothing beyond what Corollary 2 already established.

REVIEW – Corollary 3 leans entirely on the “fresh check gates every exit” property of `exit_if_idle` in `multilevel.py` – worth reading that function directly (not just this prose summary) to confirm the hold-off bookkeeping (`evidence_clear_since`) genuinely never substitutes for the fresh `_natural_level` check, including on the escalation-reset path. Also worth checking: does the scope restriction to full-exit-only (inherited from Corollary 2) actually still hold here, or does chaining a delay onto the exit introduce anything Corollary 2 didn’t have to consider?

A real, paired sweep – `exit_opportunity.hysteresis_sweep`, not a retrospective estimate – measured `hold_off` $\in \{0, 25, 50, 100, 200, 400, 800, 1600, 3200\}$ ticks against `"idle"`, shed-early, point B, across U and both deadline regimes ($n = 16$ task sets, $n = 7$ at $U = 0.9/\text{tight}$, 5 seeds). Both curves – service-ratio gain and level-transition increase – fall as `hold_off` grows, but the transition increase falls faster, which is what makes an intermediate `hold_off` worth choosing rather than either extreme. Anchoring on the smallest `hold_off` that caps the level-transition increase at $\leq 10\%$:

Table 18: Smallest `hold_off` capping the level-transition increase at 10%, and the fraction of the `hold_off=0` service-ratio gain retained at that point.

U	Regime	hold_off for LT $\leq 10\%$	Service-ratio gain retained
0.60	implicit	400	28.1%
0.60	tight	200	39.2%
0.70	implicit	400	31.5%
0.70	tight	200	43.4%
0.80	implicit	400	35.8%
0.80	tight	400	19.6%
0.90	implicit	1600	10.5%
0.90	tight	800	10.5%

The sweet spot moves with utilisation, and not in the forgiving direction. At $U = 0.6\text{--}0.7$ a modest hold-off (200–400) caps oscillation cheaply, keeping 28–43% of the gain. At $U = 0.9$ the same 10% cap needs a hold-off 2–8 $\times$ larger and keeps only $\approx 10.5\%$ – churn is denser at high utilisation (baseline level-transition counts roughly double from $U = 0.6$ to $U = 0.9$ in the same regime), so a fixed hold-off suppresses proportionally less of it. There is no single hold-off that is simultaneously cheap and effective across the whole grid tested; any deployed value is a choice about which utilisation regime to favour, not a free parameter fixed once. An interactive rendering of the full sweep (both metrics, both regimes, all four U values) is available alongside this project’s supporting documentation (`docs/exit_strategy_analysis.md`, “Result 4”).

DECISION – The data suggest `hold_off` shouldn’t really be one fixed constant – it wants to track something like current utilisation. Worth exploring: could `hold_off` be derived at runtime from an observable proxy (recent trigger rate, occupancy) rather than fixed at deploy time? That’s unimplemented and unmeasured – a genuine next phase, not a small addition. Separately, and smaller: this sweep only covers operating point B (termination); point A (conservative) hasn’t been swept, so it isn’t known whether the same U-dependence shows up there.

Progressive: cascade adds almost nothing beyond full exit alone. The diagnostic upper bound (before the mechanism existed) suggested progressive’s opportunity might exceed `shed_early`’s. Decomposing it by what kind of exit each unjustified drop would have needed answers whether that headroom is real: of the same JNE-during-tail measure, 92–97% is recoverable by full exit alone in every cell tested, leaving a cascade-specific remainder of 0.04–0.86 percentage points – itself below this study’s 5% floor everywhere, and an order of magnitude below it in most cells. Intuitively this is because a shallow rung’s evidence typically has a head start on clearing: by the time the deepest rung’s evidence has gone stale, the shallow rung’s usually has too, so the system needs a partial demotion (not a full exit) only rarely. **This closes the cascade question with evidence, the same way Stage 3 closed the drop-strategy search:** a general cascade-exit mechanism would need a new abstraction in `multilevel.py` (direct exit is currently the only exit rule it supports) and a new safety argument (unlike full exit, an intermediate level’s admissibility was certified for a system that *escalated* to it, not one that *demoted* to it – Corollary 2’s scope note), and the table above shows that investment has almost nothing left to buy. It is not recommended.

REVIEW – Lower priority than the two notes above, but worth a quick sanity check: does 0.04–0.86pp of headroom, consistently below the 0.79pp Stage 3 found for drop-strategy search, actually read as conclusive to you, or does it want a slightly larger population at the hardest cells (where progressive’s own n is already thin) before calling it closed? This is the kind of small-effect claim the project has been burned by before (the $U=0.85$ retraction).

Safety. Corollary 2 of the safety proof establishes that evidence-cleared exit changes none of Theorem 1, the LO-deadline corollary, Lemma 1, or Corollary 1’ – full exit to L_0 inherits AMC-RH’s own existing safety argument rather than needing a new one, because L_0 is unrestricted normal mode under both schemes. It is explicitly *not* claimed that total time degraded can only fall over a whole run: admitting a LO job idle-exit would have dropped can, via priority interference, pull a later HI-criticality task’s inherited busy-period start earlier and trigger an escalation the idle-exit run would not have had. Safety does not depend on this either way; net TiD, service ratio and oscillation are exactly the measured quantities in Tables 16 and 17, not derived from the proof.

REVIEW – This paragraph records a correction: an earlier draft of Corollary 2 claimed whole-run TiD can only decrease under evidence-cleared exit, which is not proven and is not true in general (the priority-interference mechanism above). It was caught while writing the proof, not after. Worth treating as a flag on this whole line of proof work rather than a closed matter – if one overclaim slipped through a first pass, a second pass (yours) is the right way to find out whether there is another.

21.6 Status of the original phases

Section 21.7 (landscape characterisation) is superseded by Stage 0 and Stage 1: it characterised severity, which is now known to be vacuous under the endorsed policy. Section 21.8 (method comparison) is retired for the reason above. Section 21.9 (budget calibration) survives intact as Stage 4, and its seeds-versus-task-sets finding still holds – it is a property of the ensemble’s variance structure, not of what is being compared. They are retained below as the record of what was planned and why it changed.

21.7 Phase A: Landscape Characterisation

Question. How does the objective actually vary with severity, and how much of that variation is signal versus noise at a practical seed budget?

Method. Sweep a coarse severity grid against a small, diverse task set ensemble (spanning the utilisation range this work studies), holding the seed block fixed across every grid point (common random numbers). This does not yet require the k -level engine: the severity-indexed trigger $R_i(\chi)$ (severity-indexed response

time, task_model.tex) can be dropped into the existing two-level engine as a single threshold, in place of R_i^{lo} , isolating one dimension of the eventual landscape – trigger *frequency* as a function of severity – without yet modelling a graded operating budget or a nested drop set. That is a real limitation, not a simplification of convenience: it characterises when a level would fire, not what Φ would be once it does, so it is necessary but not sufficient evidence for Section 18.

Pilot (executed). 16 task sets (4 at each of $U \in \{0.6, 0.7, 0.8, 0.9\}$, $n = 20$ tasks each, R1-schedulable), $\chi \in \{0, 0.05, 0.1, 0.2, 0.3, 0.5, 0.7, 1.0\}$, 24 seeds per (task set, χ) pair, 10^6 -tick horizon scaled to 4×10^5 ticks, $f_p = 5 \times 10^{-3}$: 3,072 simulation runs in 84 s (≈ 27 ms/run).

Table 19: Trigger frequency (NiD) and abandoned jobs (JNE) vs. severity, mean over the ensemble $\pm$ standard error

χ	mean NiD	mean JNE	mean TiD
0.00	7.33 ± 1.55	3.87 ± 0.71	0.00100 ± 0.00014
0.05	6.91 ± 1.49	3.31 ± 0.64	0.00085 ± 0.00013
0.10	6.60 ± 1.47	2.85 ± 0.59	0.00074 ± 0.00012
0.20	5.30 ± 1.14	1.90 ± 0.45	0.00046 ± 0.00009
0.30	4.60 ± 1.05	1.21 ± 0.33	0.00031 ± 0.00007
0.50	3.54 ± 0.93	0.41 ± 0.15	0.00011 ± 0.00003
0.70	1.39 ± 0.45	0.08 ± 0.04	0.00002 ± 0.00001
1.00	0.00 ± 0.00	0.00 ± 0.00	0.00000 ± 0.00000

Findings. (i) Mean NiD is non-increasing across all seven consecutive pairs of grid points – zero violations – which is an observable, testable consequence of property (B) (severity-indexed response time, task_model.tex) rather than an independent discovery, and it is reassuring that the data does not contradict it. (ii) At $\chi = 1$, NiD is exactly zero for every task set in this ensemble at this seed budget: charging every task uniformly at C_i^{hi} is conservative enough that, for an R1-schedulable population not further filtered for being AMC-rtb non-trivial, the deepest severity is rarely or never reached inside a 4×10^5 -tick horizon. (iii) The between-task-set coefficient of variation at $\chi = 0.1$ is **0.89** – larger than the per-seed noise reported in Section 16 – confirming that task-set-to-task-set variation, not Monte Carlo noise within a task set, is the dominant source of spread in this comparison, which is what Section 21.9 exploits directly.

Exit criterion. A characterisation this coarse cannot certify smoothness, only fail to contradict it; the finding that matters for Section 21.8 is that the signal (a several-fold change in NiD across the grid) is large relative to the per-point standard error, so a method comparison built on this ensemble is comparing something real, not noise.

21.8 Phase B: Method Comparison

Question. Under a matched evaluation budget, does any candidate method from Section 17 find a materially better configuration than exhaustive enumeration, or find a comparable one for less?

Method. Fix a total evaluation budget equal to what exhaustive enumeration spends at a chosen k (lattice size $\times$ seed budget from Section 21.9). Give a genetic algorithm and a random-search baseline the identical budget, each repeated independently several times with different internal random seeds – the metaheuristics carry their own randomness on top of the simulator’s, and a single run of either cannot distinguish a good method from a lucky one. Report, per method: regret against the enumeration’s known optimum (enumeration is exhaustive, so it is the ground truth this comparison needs, not an estimate); the variance of the reported optimum across the method’s own repeated runs; and wall-clock cost.

Status. Not yet executed. Section 21.7’s pilot uses the existing two-level engine as a stand-in for a single trigger; a real comparison needs Φ from an actual k -level run (graded operating budget, nested drop set), which needs the k -level engine this project has not yet built. This phase is specified precisely so it can run as soon as that engine exists, rather than being designed after the fact once it is built.

21.9 Phase C: Budget Calibration

Question. How many task sets and how many seeds per task set does the eventual comparison actually need – enough for the study’s stated 5% practical-significance threshold to be resolvable, not more than that?

Method. Run a small pilot, aggregate to one value per task set (`aggregate_by_taskset`), and feed the pilot into `required_pairs()` (`amc_tasksim.experiments.contract`) to estimate the ensemble size the target effect needs. Then confirm the prediction empirically – rerun at the predicted scale and check the standard error actually narrowed as predicted – *before* committing the full study’s compute budget to it.

Pilot (executed). Comparing two adjacent, hard-to-separate severities from Section 21.7 ($\chi = 0$ vs. $\chi = 0.05$) on JNE, with the same 16-task-set ensemble and 10 seeds per task set:

```
n=16 diff=-0.469 (-12.8%) 95% CI [-0.652, -0.286] resolvable=5.0% practically
significant
```

`required_pairs()` against a 5% target effect returned **54** task sets, against the pilot’s 16.

The confirmation step did not go as expected, and that is the finding. The first attempt scaled the wrong axis – seeds per task set, $10 \rightarrow 40$ – expecting the standard error to roughly halve ($1/\sqrt{4}$). It did not: the ratio measured was $0.92\times$, not $2\times$. Re-running instead with `required_pairs()`’s actual axis, task-set count ($16 \rightarrow 32 \rightarrow 64$, seeds held at 10), gave resolvable bounds of $5.0\% \rightarrow 6.4\% \rightarrow 4.5\%$ – noisy, but tracking in the right direction, unlike the seed-count attempt. Section 21.7’s between-task-set CV of 0.89 explains why directly: when between-task-set variance dominates Monte Carlo noise within a task set, adding seeds to each task set narrows only the smaller component, and the paired standard error barely moves. **The lever that narrows it is more task sets, not more seeds per task set** – the opposite of the first, more natural-seeming instinct, and not something the structural argument in Section 18 would have surfaced on its own.

Exit criterion. A calibrated (ensemble size, seed count) pair, confirmed by rerunning at that scale and checking the resolvable bound against the target – not merely computed once and trusted.

22 What This Protocol Does and Does Not Yet Establish

This section is written after Section 21, and supersedes the version of it written before the k -level engine existed. The original question – does a metaheuristic beat exhaustive enumeration at finding good trigger severities? – has been answered, but not by running that comparison. It has been answered by showing the comparison is not worth running: trigger spacing has no effect on the endorsed (shed-early) policy at all, and every knob that survives Stage 0’s pruning moves service ratio by under 1.6% against this study’s own 5% threshold, with Stage 3’s exhaustive bound placing at most 0.79% on the table for *any* method to find. Enumeration does not need to be shown to beat a genetic algorithm when the prize either method could win is an order of magnitude below what this study calls a result.

What *is* established, and at a scale the original three-phase design did not anticipate needing: the scheme beats two-level AMC-RA in all 32 configurations of the regime map, by a widening margin as utilisation rises and deadlines tighten (+2.4% to +27.3%), with 22 of those 32 results practically significant by this study’s own threshold. That is the actual positive result this line of work set out to find, and it did not require optimising trigger spacing to get it – it came from the choice of drop policy (shed-early over progressive-by-default; progressive where feasible) and operating point (conservative versus termination), which Stage 1 and Stage 2 characterise directly.

Stage 5 adds a second positive result Stages 0–4 were not designed to look for: evidence-cleared exit beats the shipped idle-only exit on service ratio in 16 of 16 cells tested, 11 practically significant, up to +27% – the largest exit-related effect this study has measured, and provably safe for the adopted shed-early policy by inheritance from AMC-RH’s own existing argument (Corollary 2), not a new one. It comes with a real, equally well-resolved cost: level transitions rise 20–87% across the same cells, growing alongside the benefit rather than independently of it. Whether the net trade is worth taking against the full objective Φ is not yet settled here (Section 21.5); what is settled is that neither number can be reported without the other. A tempered variant answers a narrower question fully: Section 21.5.1 shows a real sweet spot exists

(churn falls faster than benefit as the hold-off grows), proves it safe for any hold-off by the same inheritance argument (Corollary 3), and shows that sweet spot is not one fixed value – it needs a hold-off 2–8× larger at $U = 0.9$ than at $U = 0.6$ – 0.7 to hold the same oscillation cap, retaining a correspondingly smaller share of the gain. Progressive’s cascade-exit question, by contrast, *is* settled: decomposing the same measurement shows a real cascade mechanism could add at most 0.86 percentage points beyond full exit alone, in every cell tested – below threshold everywhere, closing that question the same way Stage 3 closed drop-strategy search, without needing to build or prove the mechanism.

Three things are not yet established and are named rather than glossed over. First, the regime map’s grid is coarse ($U \in \{0.6, 0.7, 0.8, 0.9\}$, two deadline regimes, one progressive spacing) – it is not a claim that every point in the continuous $(U, D_i/T_i, CF, f_p)$ space behaves this well, only that no point tested behaves badly, which is itself a correction of an earlier, narrower claim to the contrary (Section 21.2). Second, progressive feasibility falls as low as 50% at the hardest utilisation and deadline combinations tested, and lower still where the population itself becomes scarce ($U = 0.9$, tight); the regime map’s progressive rows are therefore conditioned on a shrinking, and not necessarily representative, subset of the population as the regime hardens. Third, Stage 5’s net verdict on evidence-cleared exit is unresolved by design, not by omission: it reports the two components of the trade honestly rather than combining them into a single number this study has not validated a weighting for. All three are stated as scope, not as doubt about the results reported within that scope.

23 Review Notes and Open Decisions

This section exists only for this review pass. Remove it, along with the `\reviewnote/\decisionnote` macro definitions and every call to them above, once the items below are resolved.

The highlighted boxes throughout Sections 21.5 and 21.5.1 (**REVIEW** and **DECISION**) each attach to a specific location. A few items don’t attach to one place, or came up in discussion around this draft rather than in the text itself; they are collected here instead of forced into a box somewhere.

23.1 Where to concentrate review time

Highest value first: the two safety corollaries (Corollary 2, Corollary 3 – `safety_proof.md`) and the mechanism code they justify (`exit_if_idle`, `_natural_level`, `_next_evidence_reappearance` in `multilevel.py`). These are the highest-stakes additions this pass produced – safety arguments about a real-time scheduling system – and this is exactly the category where the project has a track record of catching real mistakes, including one caught during this pass itself (the whole-run TiD overclaim, flagged inline above). The *mechanism* carries strong automated assurance already: `hold_off=0` and `hold_off→∞` are verified *bit-identical* to `"amc_rh"` and `"idle"` respectively, not merely similar. The *proof reasoning* has had one careful pass and one self-correction, which is evidence of care but not a substitute for an independent read.

Lower priority: the experiment functions, the reconciliation docs, and the measured data. These are backed by 361 passing tests including the equivalence checks above, and are lower-consequence if something is off – a measurement bug shows up as an implausible number, not a safety hole.

23.2 Untested combinations (proof coverage, not measurement coverage)

Corollary 2 and Corollary 3’s safety arguments are stated generally (they extend to $x_{LO} > 0$ via Corollary 1’(a)), but no empirical trial has actually run `"amc_rh"` or `"hysteresis"` with $x_{LO} > 0$ – every measured result in Section 21.5 uses $x_{LO} = 0$. If LO-criticality triggering is a direction this project wants to keep alive, that combination is unmeasured, not just unreported.

23.3 Decisions collected from elsewhere in this draft

- **The Φ net-verdict** (flagged in Section 21.5): implement Φ and re-score, or make an explicit engineering call in prose. Blocks a single recommended configuration for evidence-cleared exit.

- **Adaptive hold_off** (flagged in Section 21.5.1): the sweet spot’s dependence on U suggests **hold_off** might want to be runtime-derived rather than fixed at deploy time. Unscoped, unimplemented – a candidate for a genuine next phase of work, not an afternoon’s addition.
- **Operating point A for the hysteresis sweep**: only point B (termination) was swept; point A (conservative) is untested.

23.4 Explicitly not recommended, but worth confirming you agree

Extending "amc_rh"/"hysteresis" to the progressive policy. The cascade-headroom decomposition (Section 21.5) already suggests very little is left to gain there, on top of the open carry-in question that would make it unsafe to deploy without new proof work regardless. If you disagree with deprioritising this, say so here rather than downstream.

23.5 Housekeeping, not physics

This draft’s new content (all of Section 21.5 and this section) was written in one extended session and compiles cleanly, but has not had the iterate-and-tighten prose pass the rest of this paper has had over time. Notation, redundancy between paragraphs, and whether Section 21.5.1 is better as its own \subsection than a \subsubsection of Stage 5 are all fair game.

24 Conclusion

The central claim of this document is that graceful degradation can be added to mixed-criticality scheduling without weakening the guarantee the field already relies on: every level of the severity ladder is certified by the same response-time analysis used for the classic two-level case, generalised rather than replaced (Sections 2 and 7), and the scheme beats two-level AMC-RA across every configuration of a properly-powered regime map (Section 21). Getting there required correcting more than one claim along the way rather than only confirming them – an underpowered pilot’s apparent loss at the hardest tested regime, a retracted unconditional safety statement replaced by a graded one, an overclaim about whole-run time-degraded caught while writing its own proof – and each correction is left visible in the text rather than silently absorbed, on the view that a document which never revises itself in front of the reader is less trustworthy, not more.

What was not anticipated at the outset is where the largest remaining effect turned out to live: not in how many levels to use, or where to place their triggers – both vacuous under the adopted policy – but in *exit* timing, a question the original protocol was not designed to ask (Section 21.5). That result is reported with the same discipline as the rest of the document: a real, measured cost (increased oscillation) alongside the real, measured benefit, neither one asserted to dominate the other without a validated way to weigh them.

This is a review draft, not a finished one. Section 23 collects what still needs a decision – most of it now concentrated in exit-timing behaviour, not in the parts of the argument that were settled earlier and have stood since.

References

- [1] I. Bate, A. Burns, and R. Davis. A bailout protocol for mixed criticality systems. In *EUROMICRO Conference on Real-Time Systems*, pages 259–268, 2015.
- [2] I. Bate, A. Burns, and R. Davis. An enhanced bailout protocol for mixed criticality embedded software. *IEEE Transactions on Software Engineering*, 43(4):298–320, 2017.
- [3] I. Bate, A. Burns, and R. Davis. Analysis-runtime co-design for adaptive mixed criticality scheduling. In *Real-Time and Embedded Technology and Applications Symposium (RTAS)*, pages 187–200, 2022.

- [4] A. Burns and R. Davis. Mixed criticality systems - a review. <https://www-users.york.ac.uk/~ab38/review.pdf>.
- [5] A. Burns, R. I. Davis, S. Baruah, and I. Bate. Robust mixed-criticality systems. *IEEE Transactions on Computers*, 67(10):1478–1491, 2018.
- [6] P. Emberson and I. Bate. Extending a task allocation algorithm for graceful degradation of real-time distributed embedded systems. In *29th Real-Time Systems Symposium*, pages 270–279, 2008.
- [7] Michael C. Fu. Feature article: Optimization for simulation: Theory vs. practice. *INFORMS Journal on Computing*, 14(3):192–215, 2002.
- [8] S. Gordon and D. Whitley. Serial and parallel genetic algorithms as function optimizers. In *Fifth International Conference on Genetic Algorithms*, pages 177–183, 1993.
- [9] P. Graydon and I. Bate. Safety assurance driven problem formulation for mixed-criticality scheduling. In *1st International Workshop on Mixed Criticality Systems*, pages 19–24, 2013.
- [10] D. Griffin, I. Bate, and R. Davis. Generating utilization vectors for the systematic evaluation of schedulability tests. In *Real-Time Systems Symposium*, 2020.
- [11] Donald R. Jones, Matthias Schonlau, and William J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13(4):455–492, 1998.
- [12] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- [13] J.-C. Laprie. Dependable computing and fault tolerance: Concepts and terminology. In *15th IEEE International Symposium on Fault-Tolerant Computing*, pages 2–11, 1985.
- [14] S. Law, I. Bate, and B. Lesage. Justifying the service provided to low criticality tasks in a mixed criticality system. In *28th International Conference on Real Time Networks and Systems*, pages 100–110, 2020.
- [15] W. Nace and P. Koopman. A product family approach to graceful degradation. In *IFIP WG10.3/WG10.4/WG10.5 International Workshop on Distributed and Parallel Embedded Systems*, pages 131–140, 2001.
- [16] R. Schlichting and F. Schneider. Fail-stop processors: An approach to designing fault-tolerant computing systems. *Computer Systems*, 1(3):222–238, 1983.
- [17] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P. Adams, and Nando de Freitas. Taking the human out of the loop: A review of Bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2016.
- [18] C. Shelton. *Scalable Graceful Degradation for Distributed Embedded Systems*. PhD thesis, Department of Electrical and Computer Engineering, Carnegie Mellon University, 2003.
- [19] J. Tate, B. Woolford-Lim, I. Bate, and X. Yao. Comparing design of experiments and evolutionary approaches to multi-objective optimisation of sensor network protocols. In *10th IEEE Congress on Evolutionary Computation*, pages 1137–1144, 2009.
- [20] S. Vestal. Preemptive scheduling of multi-criticality systems with varying degrees of execution time assurance. In *28th Real-Time Systems Symposium*, pages 239–243, 2007.
- [21] K. Whitley. A genetic algorithm tutorial. Technical Report CS-93-103, 1993.